

Министерство науки и высшего образования Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности

Работа допущена к защите
Руководитель ОП
_____ А.В. Щукин
«_____» _____ 202_ г.

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
РАБОТА БАКАЛАВРА
АВТОМАТИЧЕСКАЯ ОПТИМИЗАЦИЯ КОНФИГУРАЦИИ КЛАСТЕРА
РАСПРЕДЕЛЕННОЙ БАЗЫ ДАННЫХ APACHE IGNITE
по направлению подготовки 09.03.03 Прикладная информатика
Направленность (профиль) 09.03.03_03 Интеллектуальные инфокоммуникацион-
ные технологии

Выполнил
студент гр. з5130903/00301

Ф.В. Шергалис

Руководитель
ст. преподаватель ВШ ПИ

Е.Е. Андрианова

Консультант
по нормоконтролю

В.А. Пархоменко

**САНКТ-ПЕТЕРБУРГСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
ПЕТРА ВЕЛИКОГО**

Институт компьютерных наук и кибербезопасности

УТВЕРЖДАЮ

Руководитель ОП

_____ А.В. Щукин

« _____ » _____ 202_г.

ЗАДАНИЕ

на выполнение выпускной квалификационной работы

студенту Шергалису Филиппу Витальевичу гр. з5130903/00301

1. Тема работы: Автоматическая оптимизация конфигурации кластера распределенной базы данных Apache Ignite.
2. Срок сдачи студентом законченной работы: 11.01.2025.
3. Исходные данные по работе: Ansel, Kamil, Veeramachaneni [3.1], *Apache Ignite Documentation* [3.2] и Van Aken, Pavlo, Gordon [3.3]
 - 3.1. *Ansel J., Kamil S., Veeramachaneni K.* OpenTuner: An Extensible Framework for Program Autotuning // Proceedings of the 23rd International Conference on Parallel Architectures and Compilation. — Edmonton, AB, Canada: Association for Computing Machinery, 2014. — С. 303—316. — (Сер.: PACT '14). — DOI 10.1145/2628071.2628092. — URL: <https://doi.org/10.1145/2628071.2628092>.
 - 3.2. *Apache Ignite Documentation*. — URL: <https://ignite.apache.org/docs/latest/> (дата обращения: 28.01.2025).
 - 3.3. *Van Aken D., Pavlo A., Gordon.* Automatic Database Management System Tuning Through Large-scale Machine Learning // Proceedings of the 2017 ACM International Conference on Management of Data. — Chicago, Illinois, USA: Association for Computing Machinery, 2017. — С. 1009—1024. — (Сер.: SIGMOD '17). — DOI 10.1145/3035918.3064029. — URL: <https://doi.org/10.1145/3035918.3064029>.
4. Содержание работы (перечень подлежащих разработке вопросов):

- 4.1. Проанализировать особенности конфигурации Apache Ignite и существующие решения.
- 4.2. Сформулировать технические требования, рамки MVC (Minimum Valuable Product).
- 4.3. Разработать оптимизатор.
- 4.4. Внедрить оптимизатор в командную строку Apache Ignite.
- 4.5. Протестировать получившийся программный продукт в различных конфигурациях кластеров.
- 4.6. Оценить эффективность разработанного решения на практике и определить перспективы развития.
5. Перечень графического материала (с указанием обязательных чертежей): нет.
6. Консультанты по работе:
 - 6.1. Ст. преподаватель ВШ ПИ, В.А. Пархоменко (нормоконтроль).
7. Дата выдачи задания: 01.10.2024.

Руководитель ВКР

Е.Е. Андрианова

Задание принял к исполнению

01.10.2024

Студент

Ф.В. Шергалис

РЕФЕРАТ

На 43 с., 17 рисунков, 2 таблицы, 3 приложения

КЛЮЧЕВЫЕ СЛОВА: РАСПРЕДЕЛЕННЫЕ СИСТЕМЫ, АРАСНЕ IGNITE, КОНФИГУРАЦИЯ, ОПТИМИЗАЦИЯ.

Тема исследования: «Автоматическая оптимизация конфигурации кластера распределенной базы данных Apache Ignite».

Настоящая работа посвящена исследованию автоматической оптимизации конфигурации кластера распределенной базы данных Apache Ignite. В ходе работы разработана утилита командной строки для автоматизированной настройки конфигурации с учетом особенностей оборудования, анализа хранимых данных и требований к системе.

Предмет исследования: автоматизация конфигурирования Apache Ignite.

Цель работы: разработка утилиты для упрощения администрирования Apache Ignite с акцентом на эффективное использование ресурсов.

Методы и методология: анализ существующих решений, разработка оптимизатора с использованием эвристического анализа и машинного обучения (OpenTuner), тестирование на различных конфигурациях кластеров.

Результаты: создана утилита, позволяющая автоматически анализировать и подбирать параметры конфигурации Apache Ignite, что ускоряет процесс настройки и повышает производительность.

Область применения: администрирование кластеров Apache Ignite, автоматизация настройки СУБД.

Выводы: предложенная утилита упрощает процесс конфигурации и помогает неопытным администраторам достичь оптимальной настройки системы быстрее, чем при ручной настройке.

ABSTRACT

43 pages, 17 figures, 2 tables, 3 appendices

KEYWORDS: DISTRIBUTED SYSTEMS, APACHE IGNITE, CONFIGURATION, TUNING.

Research topic: «Automated Tuning of the Distributed Database Management System Apache Ignite».

This paper is dedicated to the study of automatic optimization of the configuration of an Apache Ignite distributed database cluster. During the research, a command-line utility was developed for automated configuration tuning, taking into account hardware specifics, stored data analysis, and system requirements.

Subject of research: automation of Apache Ignite configuration.

Objective: development of a utility to simplify Apache Ignite administration with a focus on efficient resource utilization.

Methods and methodology: analysis of existing solutions, development of an optimizer using heuristic analysis and machine learning (OpenTuner), testing on various cluster configurations.

Results: a utility was created that automatically analyzes and selects Apache Ignite configuration parameters, accelerating the tuning process and improving performance.

Application area: administration of Apache Ignite clusters, automation of database management system configuration.

Conclusion: the proposed utility simplifies the configuration process and helps inexperienced administrators achieve optimal system settings faster than manual configuration.

СОДЕРЖАНИЕ

Введение	8
Глава 1. Сравнительный анализ инструментов оптимизации баз данных	11
1.1. Обзор СУБД Apache Ignite	11
1.2. Анализ существующих продуктов.....	12
1.2.1. Анализ инструмента оптимизации DB2 Advisor	12
1.2.2. Анализ инструмента оптимизации Ottertune.....	13
1.2.3. Анализ инструмента оптимизации CDBTune	13
1.2.4. Анализ инструмента оптимизации PGTune	13
1.2.5. Анализ инструмента оптимизации DBTune.....	14
1.3. Критерии для сравнения.....	14
1.4. Обзор фреймворка для оптимизации программ OpenTuner	15
1.5. Выводы	16
Глава 2. Разработка решения для автоматизированной оптимизации Apache Ignite	17
2.1. Обзор предлагаемого решения.....	17
2.1.1. Методы оптимизации.....	18
2.1.2. Диаграмма вариантов использования.....	19
2.1.3. Диаграмма состояний	19
2.1.4. Диаграмма классов	21
2.2. Минусы предлагаемого решения.....	23
2.3. Выводы	24
Глава 3. Реализация программного решения для оптимизации конфигурации Apache Ignite.....	25
3.1. Выбор технологий.....	25
3.2. Обзор исходных кодов реализации	26
3.2.1. ignite-cli	26
3.2.2. ignite-rest-api	29
3.2.3. ignite-rest	30
3.2.4. ignite-optimiser	30
3.3. Выводы	32
Глава 4. Тестирование и результаты апробации оптимизатора.....	34
4.1. Модульное тестирование	34
4.2. Интеграционное тестирование	34
4.3. Функциональное тестирование.....	35
4.4. Защита данных.....	36

4.5. Аprobация.....	36
4.6. Дальнейшее развитие.....	39
4.7. Выводы	40
Заключение	41
Список использованных источников.....	42
Приложение 1. Исходный код приложения	44
Приложение 2. Исходный код тестов приложения.....	91
Приложение 3. Конфигурации кластеров использованные при апробации .	98

ВВЕДЕНИЕ

В современном мире эффективная обработка больших объемов данных играет ключевую роль во всех сферах жизни. Способность быстро сохранять, анализировать и обеспечивать доступ к данным может означать для бизнеса, крупного и малого, разницу между успехом и банкротством. Кроме того, с переносом вычислений в облачные системы остро встает проблема отказоустойчивости и непрерывной доступности данных. Вместо небольшого числа серверов, расположенных в одном датацентре и хранящих полный набор данных, теперь все чаще используются распределенные по различным локациям виртуальные сервера (“узлы“, “ноды“), сохраняющие часть информации. Таким образом, невозможно представить себе современную облачную СУБД без гарантий предотвращения потери данных в случае сетевой ошибки или планового отключения сервера.

С ростом сложности СУБД растет и число настроек, правильная комбинация которых приведёт к достижению пиковой эффективности. Потребность в подобных оптимизациях приводит к необходимости нанимать дорогостоящих специалистов, разбирающихся в особенностях используемой компанией системы хранения и вариациях её настроек [9]. Ситуация усугубляется тем, что любая достаточно крупная компания вынуждена использовать комбинации из различных систем. Например, одна база для данных пользователей, а вторая для объемных ресурсов. В свою очередь, небольшая компания не может позволить себе узконаправленных специалистов, способных настроить каждый из видов систем хранения, что может привести к неоптимальному использованию СУБД. Для решения указанной проблемы появились подходы и программы для автоматизации конфигурирования баз данных.

В данной работе объектом исследования являются Системы Управления Базами Данных. Предметом выбрана автоматизация конфигурирования распределенной СУБД Apache Ignite.

Цель данной работы – разработка утилиты для упрощения администрирования СУБД Apache Ignite с упором на оптимальное использование ресурсов системы.

Для достижения цели поставлены следующие задачи:

- А. Изучить научную литературу по теме оптимизации баз данных.
- В. Проанализировать особенности конфигурации и управления Apache Ignite и сопутствующих технологий.

- C. Сформулировать технические требования, рамки MVP (Minimum Viable Product)
- D. Написать программный код утилиты.
- E. Протестировать получившийся продукт в различных конфигурациях кластеров.
- F. Оценить эффективность разработанного решения на практике и определить перспективы развития.

Актуальность исследования обосновывается отсутствием утилит, позволяющих автоматически оптимизировать конфигурацию Apache Ignite, что затрудняет использование данной СУБД. Разработка инструмента, упрощающего настройку, позволит ускорить процесс конфигурирования, повысить производительность и снизить риски ошибок. Apache Ignite и её коммерческая версия GridGain активно используются крупнейшими международными компаниями, такими как IBM, American Airlines, Hewlett Packard Enterprise и J.P.Morgan [10]. Система Platform V DataGrid, разработанная Сбербанком на базе Apache Ignite, используется во внутренних сервисах. Предложенная утилита найдёт применение как среди профессионалов, так и среди энтузиастов, стремящихся изучить лучшие практики оптимизации баз данных.

Методологическая база исследования включает анализ современных научных подходов к проектированию и оптимизации систем управления базами данных. Перед началом разработки был выполнен сравнительный анализ существующих на рынке решений.

В ходе оценки научной разработанности проблемы выяснилось, что существующие на рынке решения обладают значительными недостатками. Традиционные инструменты полагаются на предварительно заложенные разработчиками эвристики, что сводит их возможности к ограниченному своду правил и не позволяет выйти за рамки одной СУБД. В последние годы проявилась тенденция использования для конфигурирования баз данных машинного обучения, что привело к появлению платформонезависимых решений, подходящих для оптимизации множества СУБД. Тем не менее, в основном такие инструменты не готовы к немедленному использованию, требуют дополнительных настроек и на данном этапе являются скорее результатами научных работ, а не законченных продуктов [17]. Таким образом, новизна работы подтверждается отсутствием на рынке готовых к немедленному использованию программ для конфигурации системы Apache Ignite.

Практическая значимость заключается в появлении не имеющей прямых аналогов утилиты, расширяющей функционал СУБД Apache Ignite. Так как Apache Ignite является проектом с открытым исходным кодом, возможна дальнейшая интеграция получившейся утилиты в основную кодовую базу.

Для апробации результатов был проведен эксперимент с участием нескольких добровольцев, для сравнения опыта настройки кластеров Apache Ignite с использованием данного инструмента и без него.

ГЛАВА 1. СРАВНИТЕЛЬНЫЙ АНАЛИЗ ИНСТРУМЕНТОВ ОПТИМИЗАЦИИ БАЗ ДАННЫХ

1.1. Обзор СУБД Apache Ignite

Apache Ignite разработана в 2014 году компанией GridGain, и представляет собой распределенную систему управления баз данных с открытым исходным кодом. Изначально спонсируемая Сбербанком, со временем система вышла на международный рынок и стала частью экосистемы Apache. На основе бесплатной версии, компания GridGain представила и коммерческую модификацию, обладающую расширенными возможностями. Вскоре, Сбербанк выпустил свою версию под названием Platform V DataGrid. На сегодняшний день она используется в 127 сервисах банка [12]. DataGrid также предлагает улучшенный функционал и техническую поддержку, но ориентирован на российский рынок. В 2022 году компания начала сотрудничество с Digital Design для импортозамещения своими разработками ушедших из России решений [18]. Таким образом, можно говорить о перспективности и важности технологии и в Российской Федерации, и за её пределами.

В ходе разработки были приняты решения, тормозящие дальнейшее развитие, и с 2018 года компания GridGain работает над полной переработкой системы, Apache Ignite 3. Именно эта версия будет объектом оптимизации, т.к. разрабатывать инструменты под уходящую в прошлое версию не имеет смысла. Данный фы приводит к значительной проблеме. Третья версия нестабильна и её возможности на данный момент ограничены по сравнению с предыдущей.

По умолчанию система работает в режиме In-Memory, сохраняя информацию в оперативную память и выгружая её на диск перед отключением узла. Такой подход приводит к потере данных в случае аварийной остановки, и по необходимости администратор базы может включить Persistent Storage, превращая систему в традиционную базу данных с гарантией сохранения информации.

Сложность системы приводит к обилию настроек, оптимальные значения которых отличаются в зависимости от сценариев использования кластеров, а в некоторых случаях и неизвестны из-за отсутствия достаточного понимания их влияния на работу системы. Простой анализ текстового файла со списком настроек требует глубокого понимания системы для определения значений, подходящих под нужды пользователей и оптимальной работы на используемом на серверах железе.

Кроме того, документация по администрированию СУБД требует длительного времени для изучения, и, что не менее важно, устаревает в процессе выхода новых версий из-за невозможности отслеживать все изменившиеся части [4].

Принимая прямое участие в разработке системы, автор исследования знает, что стандартные значения конфигурации в некоторых случаях определяются интуитивно, и сам добавлял новые настройки, удлинив и так значительный список. Из всего вышесказанного можно сделать вывод о важности инструмента для автоматизации процесса тестирования различных комбинаций конфигураций для данного продукта.

1.2. Анализ существующих продуктов

На момент исследования в рейтинге СУБД зарегистрировано 423 системы [8], от используемых по всему миру гигантов, до систем, не имеющих собственного сайта. В данном списке Apache Ignite находится на 90 месте, его коммерческая версия GridGain занимает 150 позицию. Крупнейшие базы данных за годы существования уже обзавелись собственными утилитами для оптимизации, но для более нишевых продуктов такой инструментарий отсутствует. Рассмотрим инструменты, поддерживающие только одну систему по своей природе, а также универсальные утилиты.

1.2.1. Анализ инструмента оптимизации DB2 Advisor

Программа DB2 Advisor разработана компанией IBM и используется для оптимизации базы данных DB2. Рисунок 1.1 демонстрирует интерфейс проекта. Эта утилита анализирует запросы к базе и по результатам рекомендует пользователю создание индексов, материализованных представлений и перераспределение партиций [7; 13]. Возможности утилиты ограничены, для работы ей необходим готовый список запросов, который приходится составлять вручную. Нет возможности автоматически применить рекомендации, хотя можно задать файл для вывода SQL запросов, создающих рекомендованные сущности. Оптимизация конфигурации не производится. Тем не менее, инструмент интегрирован в DB2 и поддерживается уже больше 20 лет.

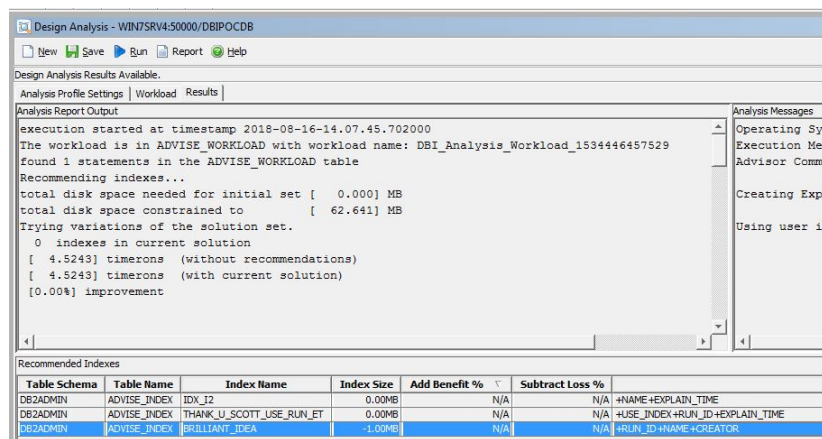


Рис.1.1. Интерфейс DB2 Advisor [7]

1.2.2. Анализ инструмента оптимизации *Ottertune*

Проект Ottertune вырос из исследования, проведенного в 2018 году [15] и использует байесовскую оптимизацию. На рисунке 1.2 продемонстрирован интерфейс утилиты. В статье указана поддержка любых баз данных, но в коммерческом продукте, который был запущен впоследствии, используется только PostgreSQL, Amazon RDS, Aurora и MySQL [11]. В 2021 году стартап привлек 2.5 миллиона долларов инвестиций. Ottertune способен оптимизировать как конфигурацию, так и анализировать запросы, рекомендуя способы для их оптимизации. За время написания данного исследования проект успел закрыться, завершив свою историю 15 июня 2024 года.

1.2.3. Анализ инструмента оптимизации *CDBTune*

В рамках очередного исследования был представлен CDBTune [16], основанный, по заявлениям авторов, на более эффективных принципах, чем Ottertune. Поддерживает автоконфигурацию и множество баз данных. Тем не менее, проект не обновлялся с момента публикации статьи, и примеры его использования найти не удалось [6]. Данный пример иллюстрирует проблемы исследовательских проектов, использующих прорывные методы, но не всегда приводящих к их внедрению и коммерческому использованию.

1.2.4. Анализ инструмента оптимизации *PGTune*

Программа PGTune разрабатывалась в 2009-2020 годах [14], есть обновляемый под новые версии PostgreSQL форк. Форк - ответвление от основного проекта, обычно под другим авторством. Утилита конфигурирует базу в зависи-

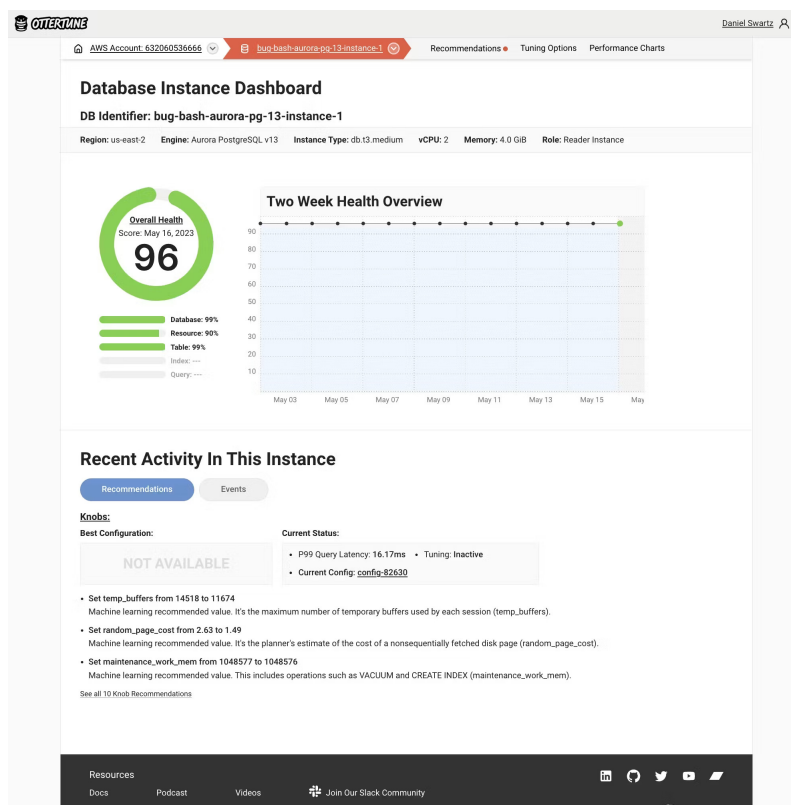


Рис.1.2. Интерфейс Ottertune [11]

мости от введенных пользователем характеристик сервера и типа приложения, не анализируя настоящую нагрузку и полагаясь на определенный набор заранее заложенных эвристик. Потенциал ограничен малыми возможностями и только одной поддерживаемой базой.

1.2.5. Анализ инструмента оптимизации DBTune

DBTune революционировал подход к оптимизации БД, предложив термин Optimiser as a service. Утилита отправляет данные об использовании базы данных на сервер с запущенным оптимизатором. Оптимальная конфигурация формируется в результате постоянного анализа нагрузки с использованием Reinforcement Learning. Тем не менее, проект на момент исследования поддерживает только PostgreSQL и обладает закрытым исходным кодом, что не позволяет поддерживать новые базы данных без содействия авторов [1].

1.3. Критерии для сравнения

В связи с обилием используемых баз данных, список поддерживаемых инструментов СУБД является ключевой характеристикой. Некоторые программы

могут быть использованы в нескольких системах, рамки применимости других ограничены одной базой.

Кроме того, отличается и предмет оптимизации. Один инструмент может работать только с логической и физической структурой базы, другие анализируют конфигурацию, третьи универсальны и оптимизируют базу на нескольких уровнях сразу.

Ранние оптимизаторы полагались на эвристики, предварительно заложенные в них разработчиками. С развитием Data Science на первый план выходит использование Machine Learning для автоматического подбора параметров.

Немаловажным критерием является и подход к оптимизации. Простые модели полагаются на серию вопросов-ответов к администратору базы данных, более продвинутые проводят бенчмарки для определения оптимальных настроек. Бенчмарк - набор тестов, измеряющих определенные параметры, обычно связанные с производительностью системы. Есть и модели, проводящие постоянный мониторинг метрик, выдаваемых базой данных в ходе работы.

Также отметим, что некоторые из приведенных инструментов - результат научных статей, и их развитие остановилось вскоре после публикации. Коммерческие же решения могут обладать более скромными характеристиками вследствие использования проверенных методов, тем не менее они поддерживаются в течение множества лет и, следовательно, обладают большим потенциалом.

Исходя из выделенных признаков и проведенного анализа существующих решений была составлена таблица 1.1.

Таблица 1.1

Таблица критериев для сравнения

Название	Базы	Предмет	Обучение	Подход оптимизации
DB2Advisor [13]	DB2	Лог/физ.	Эвристики	Анализ запросов
Ottertune [15]	Любая	Конфигурация	Байесовская	Бенчмарки
CDBTune [16]	Любая	Конфигурация	Reinforcement	Бенчмарки
PGTune [14]	PostgreSQL	Конфигурация	Эвристики	Вопрос/Ответ
DBTune [1]	PostgreSQL	Конфигурация	Reinforcement	Бенчмарки

1.4. Обзор фреймворка для оптимизации программ OpenTuner

В ходе анализа существующих решений автором был сделан вывод о перспективности использования машинного обучения для решения поставленной

задачи. Для того, чтобы воспользоваться уже существующими наработками в данной сфере, был выбран фреймворк для создания автотюнеров OpenTuner.

Фреймворк использует комбинацию техник поиска оптимальных значений, в зависимости от их успешности выбирая самый эффективный. Фреймворк используется, например, в StreamJIT для оптимизации компиляции Java кода [5] и прохождения игры Марио [3].

Это не готовый продукт, а каркас для оптимизации любой программы - от простого скрипта до СУБД или компилятора, у которой есть текстовая конфигурация и способ замерить производительность. Для создания собственного оптимизатора на базе OpenTuner необходимо разработать скрипт, генерирующий конфигурацию по переданным оптимизатором параметрам, а также скрипт для запуска бенчмарков с созданной конфигурацией и сравнения полученных результатов.

1.5. Выводы

Ни одно из найденных при анализе рынка решений не позволит настроить Apache Ignite. DB2 Advisor, PGTune и DBTune заточены под конкретные БД, Ottertune - закрытый проект, CDBTune требует провести тяжелую работу по адаптации под конкретную базу данных. Кроме того, существующие решения ограничены оптимизацией конфигурации или рекомендациями по улучшению логической структуры хранения данных. Только DB2 Advisor интегрирован в БД, остальные являются самостоятельными продуктами и требуют затрат на рекламу и продвижение. Развитие Machine Learning привело к доминированию на рынке именно AI-ориентированных решений.

ГЛАВА 2. РАЗРАБОТКА РЕШЕНИЯ ДЛЯ АВТОМАТИЗИРОВАННОЙ ОПТИМИЗАЦИИ АРАШЕ IGNITE

2.1. Обзор предлагаемого решения

В рамках анализа существующих решений сделан вывод об отсутствии подходящих инструментов для оптимизации кластера Apache Ignite. Разрабатываемое решение должно позволить произвести оптимизацию, а также замерить производительность кластера до и после применения предложенных оптимизаций.

Вместо создания отдельного продукта, устанавливаемого отдельно, выбран путь интеграции утилиты прямо в существующую утилиту для администрирования кластера. Таким образом, пользователю не придется разбираться с установкой продукта, а также в будущем возможно внедрение продукта прямо в основную кодовую базу Apache Ignite. Это возможно благодаря использованию лицензии Apache Commons, относящейся к категории "свободного ПО поощряющей модификацию и свободное распространение кода.

Так как универсальной “оптимальной“ конфигурации, которая позволит эффективно использовать любой кластер под любой нагрузкой> не существует, подходящие настройки зависят от сценариев использования кластера. пользователь должен иметь возможность выбрать бенчмарк для запуска и характеристики планируемой нагрузки на СУБД.

Кроме того, для дальнейшего анализа пользователь должен иметь возможность вывести все результаты оптимизаций и запуска бенчмарков.

Запуск оптимизации и бенчмарков может занимать длительное время, поэтому необходимо поддержать асинхронное выполнение данных команд, позволяющее выполнять другие команды до завершения процесса.

Выбор консольного представления обусловлен тем, что администрирование и эксплуатация кластеров Apache Ignite производится в первую очередь через командную строку, графический клиент используется далеко не всеми пользователями. Кроме того, использование командного интерфейса позволяет упростить разработку, тестирование и дальнейшую интеграцию в экосистему.

Так как запуск оптимизатора может повлиять на работу сервера, необходимо исключить возможность использования утилиты без соответствующих прав. Механизм проверки прав уже встроен во все компоненты Apache Ignite, что позволяет не беспокоиться о защите данных в рамках данной работы.

2.1.1. Методы оптимизации

На данном этапе развития Apache Ignite в нем отсутствует множество инструментов администрирования, давно поддерживаемых в более устоявшихся СУБД. Apache Ignite не предоставляет метрик, позволяющих администратору следить за работой системы, и которые могли бы использоваться для автоматического анализа нагрузки на СУБД. Кроме того, список доступных пользователю настроек сильно ограничен и меняется в ходе развития базы. Объем документации уступает предыдущей версии, и её обновления не поспевают за вносимыми изменениями [4]. В системе отсутствует и информация об истории запросов, аналогичной `pg_stat_activity` в PostgreSQL и позволяющей анализировать запросы к базе данных.

С учетом данных ограничений, Оптимизация конфигурации Apache Ignite может выполняться двумя основными методами:

- Эвристический анализ – поиск потенциальных проблемных мест в настройках кластера на основе заранее определённых правил.
- Автоматический подбор параметров с помощью OpenTuner – запуск серии тестов с различными конфигурациями и выбор наилучшего варианта.

По результатам анализа документации Apache Ignite были выведены следующие эвристики. Их список может быть дополнен при дальнейшем развитии программы для повышения эффективности:

- Использование памяти JVM: если система использует более 90 процентов доступной памяти, рекомендуется увеличить лимит.
- Настройки потоков GC: если число потоков для сборки мусора составляет менее половины от доступных, рекомендуется его увеличить.
- Время жизни устаревших данных: если JVM использует больше 90 процентов выделенной памяти и время жизни устаревших данных (необходимы для синхронизации транзакций в рамках распределенной СУБД) превышает 16 минут, рекомендуется его уменьшить.
- Интервал обновления LowWatermark: если превышает половину времени жизни, что приводит к редкому сбору мусора, рекомендуется его уменьшить.
- Сумма размеров регионов хранения данных, использующих оперативную память, превышает выделенное под JVM. Необходимо уменьшить, иначе при полном заполнении регионов будет ошибка Out Of Memory.

- Использование RocksDB (оптимален для записи) для read-intensive приложений или использование AIPersist (оптимален для чтения) для write-intensive приложений. Необходимо использовать подходящий движок.

2.1.2. Диаграмма вариантов использования

На рисунке 2.1 приведен разбор вариантов использования утилиты. При взаимодействии с инструментом планируется только один актер - пользователь, администратор СУБД. Основным его инструментом является запуск оптимизации. До и после изменения конфигурации имеет смысл запустить несколько бенчмарков, для сравнения результатов. Отдельная команда позволяет получить результаты оптимизации и результаты запуска бенчмарков. Также в командной строке Apache Ignite уже встроена команда для просмотра и изменения конфигурации. Кроме того, пользователь может вручную изменить файл с конфигурацией и перезапустить узел.

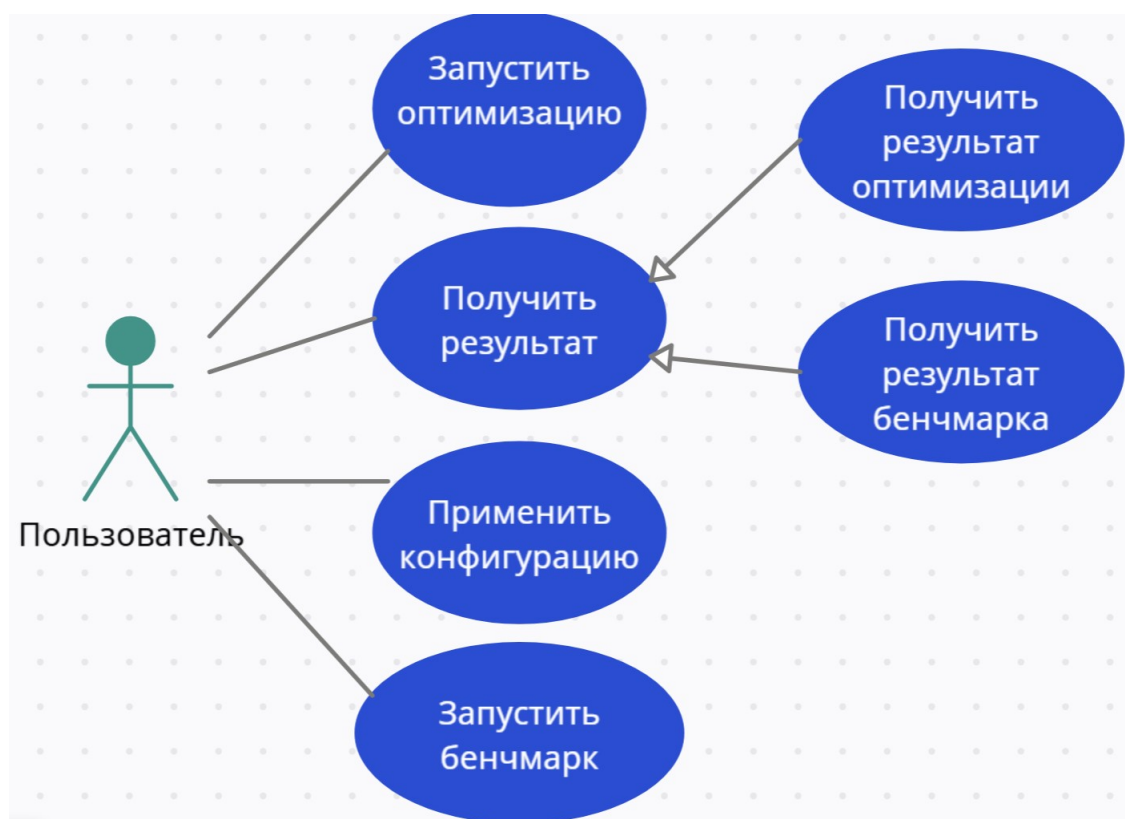


Рис.2.1. Диаграмма вариантов использования

2.1.3. Диаграмма состояний

Рассмотрим возможные изменения состояний утилиты, приведенные на рисунке 2.2. При первом запуске командной строки кластер Apache Ignite не

инициализирован. После инициализации можно сразу же запустить оптимизацию. При успешном запуске будет получен ID операции, по которому можно получить результаты. Далее пользователь применяет рекомендуемые изменения. После запуска бенчмарка также выводится ID операции, запрос результата с переданным ID покажет время выполнения.

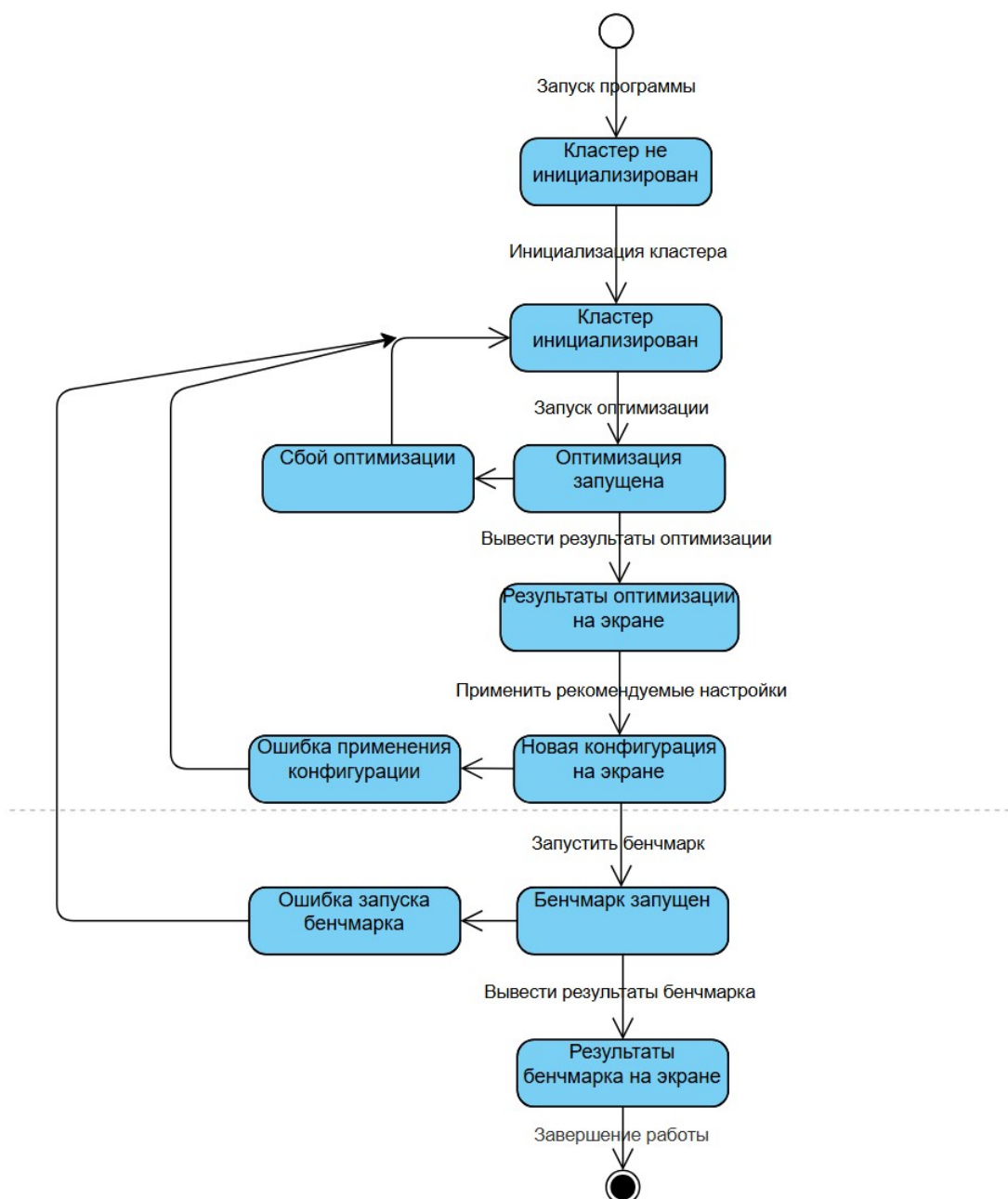


Рис.2.2. Диаграмма состояний

2.1.4. Диаграмма классов

Классы, относящиеся к исходному коду Apache Ignite и OpenTuner, в данной работе не рассматриваются. Проект состоит из нескольких ключевых компонентов, рассмотрим сначала объекты, относящиеся к командной строке.

На рисунке 2.3 представлена диаграмма классов командной строки. В классах с постфиксами Command описываются собственно аргументы команды и некоторые особенности её выполнения. Постфикс Repl обозначает, что данная команда будет использоваться в интерактивном режиме - когда пользователь подключен к конкретному узлу и вызывает команды на нём, не передавая URL при каждом вызове команды. Кроме того, каждая команда содержит соответствующий объект Call, которому передается управление после обработки пользовательского ввода. Аргументы команд вынесены в классы Mixin - группа аргументов, которую можно переиспользовать. В данном случае это необходимо для того, чтобы не дублировать наборы между Repl и обычной вариацией команды.

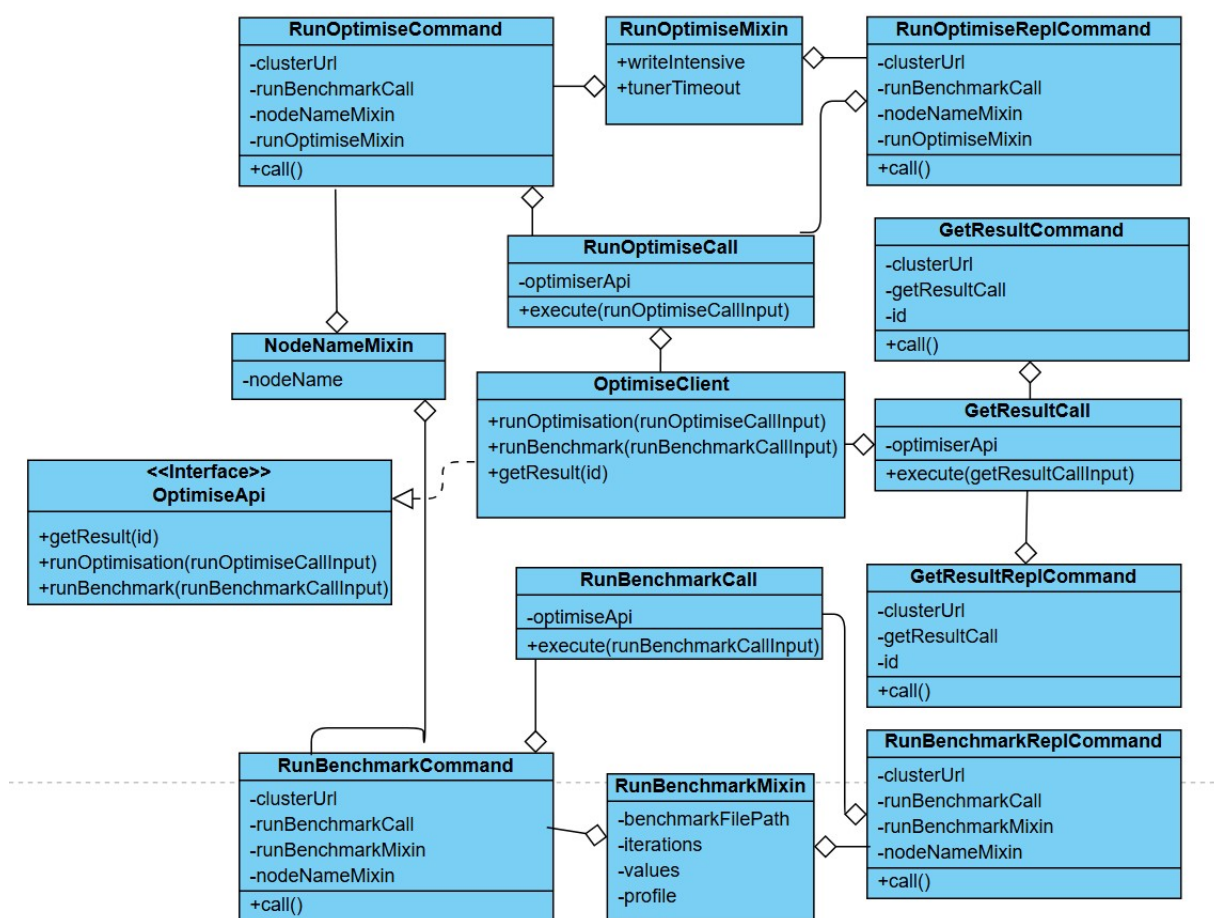


Рис.2.3. Диаграмма классов командной строки

Классы Mixin содержат только параметры и методы для того, чтобы их установить или получить. `GetResultMixin` содержит только ID операции. `RunBenchmarkMixin` содержит путь к бенчмарку, название узла, на котором необходимо его выполнить (вынесен в `NodeNameMixin` для переиспользования с командой для запуска оптимизатора), число итераций, число значений для вставки (используется если не указан файл бенчмарка) и название профиля для запуска бенчмарка - необходимо для того, чтобы сравнить скорость работы RocksDB и AIPersist без перенастройки системы. `RunOptimiseMixin` также содержит `NodeNameMixin`, логическое значение `writeIntensive` (должна ли конфигурация быть нацелена на использование при частой записи или интенсивном чтении) и численное `tunerTimeout` (сколько времени должен работать `OpenTuner`).

Постфикс `Call` используется для описания логики вызова REST контроллеров при использовании команды. Каждый из классов-вызовов содержит соответствующий клиент для отправки запросов по заданной REST-спецификации. Метод `Execute` принимает соответствующий `CallInput`, чтобы определить параметры вызова REST-метода.

Классы с постфиксом `CallInput` определяют входные данные для вызова и создаются в соответствующей команде. Они содержат только методы, связанные с установкой и получением значений параметров.

Рассмотрим логику при получении запроса от командной строки. Диаграмма классов представлена на рисунке 2.4. При вызове контроллера из команды создается соответствующий `Request`. Данные классы будут генерироваться автоматически по спецификации `openapi`.

Дальше переходим к логике на узле. Полученные запросы обрабатываются контроллером. Его задача проста - передать аргументы далее, в менеджер оптимизации. Из-за того, что пользователь может пожелать обрабатывать запросы не на том узле, к которому подключена командная строка, необходимо послать сообщение (определяются классами с постфиксом `Message`) на указанный узел. Каждый узел в кластере отслеживает данные сообщения и выполняет соответствующие методы.

Все описанные ранее объекты служат для коммуникации между пользователем и узлами кластера. Классы `OptimiserRunnerImpl` и `BenchmarkRunnerImpl` отвечают за процесс оптимизации и запуска бенчмарка соответственно.

`BenchmarkRunner` запускает бенчмарки используя методы Apache Ignite, поэтому дальнейшие взаимодействия рассматривать не будем. Класс `OptimiserRunner`

взаимодействует с конфигурацией Apache Ignite и вызывает Python программу `apachetuner.py` для запуска OpenTuner.

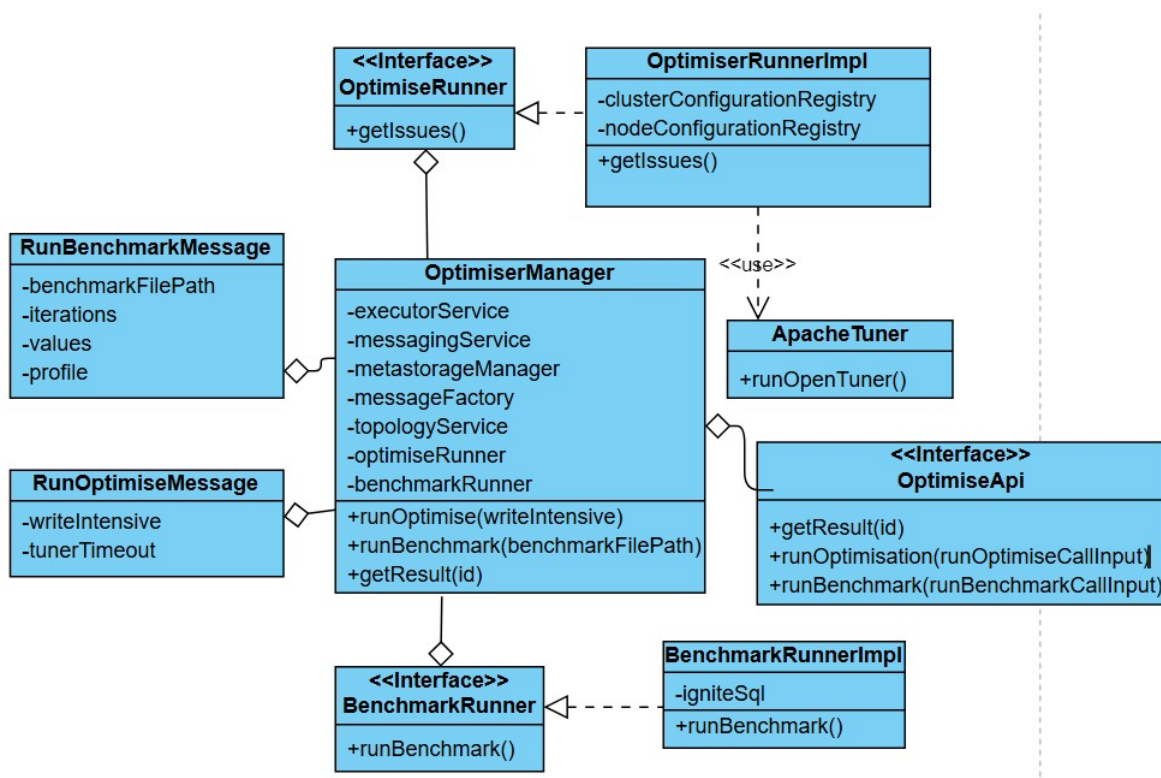


Рис.2.4. Диаграмма классов внутренней логики

2.2. Минусы предлагаемого решения

- Выбранный путь интеграции в существующий код усложняет дальнейшее развитие утилиты. В случае, если в ходе разработки Apache Ignite произойдет изменение структуры проекта, придется решать возникшие конфликты.
- Ограничения, связанные с использованием находящейся в процессе разработки версии Apache Ignite 3, не позволяют реализовать некоторые приемы, используемые при оптимизации баз данных другими инструментами. Следовательно, на данном этапе потенциал оптимизатора будет сильно ограничен. До тех пор, пока эти проблемы не будут решены при дальнейшем развитии системы.
- Внедрение внешнего инструмента оптимизации OpenTuner приводит к необходимости пользователю устанавливать дополнительную программу для использования всех возможностей оптимизатора. При доработ-

ке утилиты возможно добавить автоматическую установку и настройку OpenTuner.

- На данном этапе оптимизатор не применяет получившиеся конфигурации автоматически и требует от пользователя понимания как данные рекомендации использовать.

2.3. Выводы

Необходимо разработать систему, интегрированную в командную строку Apache Ignite, способную анализировать переданные пользователем характеристики нагрузки на кластер, текущую конфигурацию, используемые кластером ресурсы. В ходе оптимизации должен происходить запуск OpenTuner (если он установлен на узле), производящего поиск оптимальной конфигурации используя заранее созданные бенчмарки. Также пользователю должна быть предоставлена возможность замерить производительность путем запуска переданных SQL запросов.

ГЛАВА 3. РЕАЛИЗАЦИЯ ПРОГРАММНОГО РЕШЕНИЯ ДЛЯ ОПТИМИЗАЦИИ КОНФИГУРАЦИИ APACHE IGNITE

3.1. Выбор технологий

И интерфейс командной строки Apache Ignite, и сама система написаны на Java. В современном мире повышенное потребление оперативной памяти данным языком не является большой проблемой - ОЗУ дешевет, и для крупных компаний стали нормой сервера с сотнями гигабайт памяти.

Взаимодействие с разработанной системой будет осуществляться с помощью командной строки. Несмотря на наличие графического интерфейса для администрирования кластеров Apache Ignite, он является вспомогательным инструментом для более удобной визуализации данных о кластере, и не может считаться основным способом взаимодействия с кластером. При дальнейшем развитии и внедрении системы возможно добавить его поддержку и в GUI (Graphical User Interface, графический интерфейс), но подобная модификация выходит за рамки данной работы.

Для создания командной строкой используется библиотека Picosli, поддерживающая все языки, работающие на базе JVM. Особенностью данной библиотеки является простота разработки командного интерфейса с поддержкой автодополнения, различных синтаксисов - POSIX, GNU, MS-DOS и так далее, а также автоматическая генерация документации к командам.

Все взаимодействие с системой построено на принципах неблокирующего асинхронного программирования, реализованного посредством пакета Java Concurrency. Благодаря механизму CompletableFuture возможно построение системы, оптимально использующей все доступные приложению потоки и работающей без зависаний и ожиданий. Например, при отправке запроса на оптимизацию, поток, обрабатывающий запросы от REST API, только создает сообщение о необходимости оптимизации - сама логика сетевого взаимодействия обрабатывается в фоновом режиме отдельными потоками, выделенными для данной цели. Таким образом, командная строка пользователя заблокируется только на время, необходимое для обмена REST запросами между клиентом и сервером, даже если сам процесс займет часы. Результаты оптимизации можно получить отдельной командой в любой момент.

В теоретической части был проведен краткий обзор фреймворка для оптимизации программ OpenTuner. Данная система была опубликована ещё в 2014 году, но до сих пор получает обновления, имеет 404 звезды на Github и 121 форк [2], что позволяет сделать вывод об её актуальности и дальнейших перспективах.

Для использования OpenTuner требуется написать скрипт на языке Python, запускающий бенчмарк с различными вариациями конфигурации и возвращающий результат запуска. В рамках “proof-of-concept” для проведения бенчмарка был переработан уже существующий класс `UpsertKvBenchmark`, принимающий значение `raft.fsync` и проводящий вставку в поднятые специально для теста узлы Apache Ignite различных значений.

Таким образом, при запуске оптимизации Apache Ignite должен запустить данный скрипт, дождаться его завершения и вывести пользователю получившуюся конфигурацию.

3.2. Обзор исходных кодов реализации

Для внедрения утилиты в структуру Apache Ignite был создан форк репозитория. Обзор кода Apache Ignite выходит за рамки данной работы. Рассмотрим систему начиная от пользовательского интерфейса (командной строки) и дальше углубимся в собственно реализацию оптимизатора. В данной главе будут приведены только небольшие фрагменты исходного кода, полный код проекта без сокращений приведен в приложении 1.

3.2.1. *ignite-cli*

Данный модуль содержит все файлы, относящиеся к реализации командной строки. На рисунке 3.1 представлено содержимое модуля. Для удобства все реализованные в рамках данной работы классы были вынесены в отдельный пакет.

В пакет `command` помещены классы команд: запуска оптимизации, бенчмарка и получение результатов запусков, а также соответствующие им `Mixin`.

Каждая команда определяет принимаемые ею аргументы с помощью аннотации `@Option` или `@Parameters` (аргументы команды) или `@Mixin` - для классов, содержащих аргументы, переиспользуемые между несколькими командами. Для аргумента необходимо определить его название (или индекс в случае `@Parameters`) и описание для автогенерации документации. Если не указано иное, все аргумен-

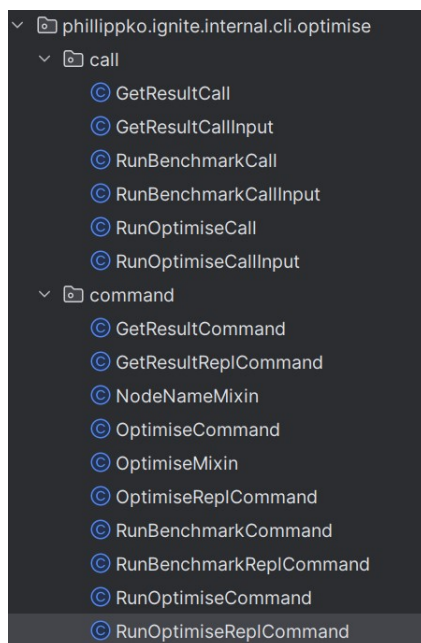


Рис.3.1. Содержимое модуля cli

ты считаются необязательными, можно установить значение по умолчанию. На рисунке 3.2 представлен пример вывода помощи.

```
[defaultNode]> optimiser result --help
Unmatched arguments from index 0: 'optimiser', 'result', '--help'
Did you mean: optimise or cluster or recovery?
[defaultNode]> optimise result --help

USAGE
  optimise result [OPTIONS] <id>

DESCRIPTION
  Prints results of the optimisation or benchmark.

PARAMETERS * - required parameter
  *   <id>          ID of the target optimisation or benchmark

OPTIONS * - required option
  --url=<clusterUrl> URL of cluster endpoint. It can be any node URL. If not set, then the
                    default URL from the profile settings will be used
  -h, --help          Show help for the specified command
  -v, --verbose        Show additional information: logs, REST calls. This flag is useful for
                    debugging
[defaultNode]> optimise --help

USAGE
  optimise [OPTIONS] [COMMAND]

DESCRIPTION
  Tools for optimising cluster configuration.

OPTIONS * - required option
  -h, --help          Show help for the specified command
  -v, --verbose        Show additional information: logs, REST calls. This flag is useful for debugging

COMMANDS
  result      Prints results of the optimisation or benchmark.
  runOptimise Optimise cluster configuration.
  runBenchmark Shows node configuration
[defaultNode]> |
```

Рис.3.2. Пример вывода помощи по использованию команд

Дальше в команде определен метод `call`, конфигурирующий поведение команды. В данном случае все команды используют стандартный вывод в терминал, при необходимости возможно назначить “декораторы” - методы, преобразующие полученный результат, например создавая таблицу.

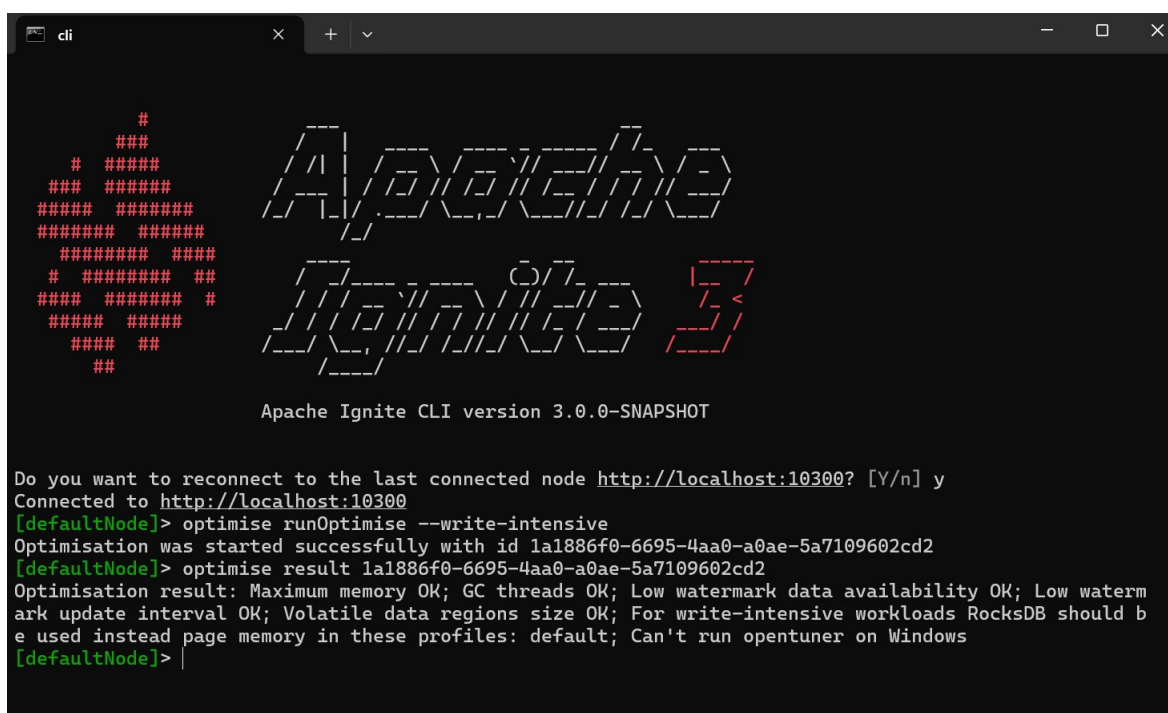
Пакет `call` включает в себя все классы "вызова" и соответствующие им параметры ввода. Их логика довольно проста. Метод `execute` принимает аргументы (помещенные в классы-обертки `CallInput`), вызывает автоматически сгенерированный по определённому в пакете REST-API клиент с полученными значениями и интерпретирует результат.

Для классов `CallInput` используется паттерн Builder, позволяющий избежать проблемы разрастающихся конструкторов в случае с большим количеством аргументов. Набор параметров в каждом `CallInput` соответствует набору аргументов в команде.

Общие аргументы:

- A. `-nodeName`, название узла, на котором необходимо выполнить оптимизацию, по умолчанию совпадает с узлом, который получил команду.
- B. `-url`, адрес узла, к которому подключается клиент.

На рисунке 3.3 изображен вызов запуска оптимизации. Аргументы, специфичные для `RunOptimiser`:



```
cli
#
###
# #####
### #####
##### #####
##### #####
##### #####
# #####
### #####
##### #####
###
##

Apache Ignite CLI version 3.0.0-SNAPSHOT

Do you want to reconnect to the last connected node http://localhost:10300? [Y/n] y
Connected to http://localhost:10300
[defaultNode]> optimise runOptimise --write-intensive
Optimisation was started successfully with id 1a1886f0-6695-4aa0-a0ae-5a7109602cd2
[defaultNode]> optimise result 1a1886f0-6695-4aa0-a0ae-5a7109602cd2
Optimisation result: Maximum memory OK; GC threads OK; Low watermark data availability OK; Low watermark update interval OK; Volatile data regions size OK; For write-intensive workloads RocksDB should be used instead page memory in these profiles: default; Can't run opentuner on Windows
[defaultNode]> |
```

Рис.3.3. Пример запуска оптимизации и получение результата

- A. `-writeIntensive`, должна ли полученная конфигурация быть оптимизирована для записи, в противном случае - для чтения.
- B. `-tunerTimeout`, время в миллисекундах, отведенное на запуск OpenTuner.

На рисунке 3.4 представлен пример запуска бенчмарка. Аргументы, специфичные для `RunBenchmark`:

- A. `–benchmarkFilePath`, путь к файлу, содержащем SQL запросы, которые необходимо запустить в рамках бенчмарка.
- B. `–iterations`, число итераций бенчмарка. В результате будет возвращено среднее значение между запусками.
- C. `–values`, используется если файл бенчмарка не задан, тогда в качестве теста используется вставка указанного числа значений в пустую таблицу.
- D. `–profile`, название профиля который необходимо использовать при запуске бенчмарка (по умолчанию используется default профиль), необходимо для сравнения RocksDB и aipersist без перенастройки сервера.

```
[defaultNode]> optimise runBenchmark C:\Users\shergalis2.fv\Downloads\ucheba\NIR\files\benchmark.sql
Benchmark was started successfully with id 6d7f7814-0ed1-4426-bdd3-f3f67d27fbeb
[defaultNode]> optimise result 6d7f7814-0ed1-4426-bdd3-f3f67d27fbeb
Optimisation result: Benchmark C:\Users\shergalis2.fv\Downloads\ucheba\NIR\files\benchmark.sql finished in 6005MS
```

Рис.3.4. Пример запуска бенчмарка и получение результата

Аргументы `GetResult`:

- A. `–id`, идентификатор операции, чей результат необходимо получить.

3.2.2. *ignite-rest-api*

В связи с несовершенностью структуры исходного проекта, невозможно создать отдельный пакет для классов, созданных в рамках ВКР, поэтому они помещены в общую с файлами Apache Ignite директорию. Содержимое модуля представлено на рисунке 3.5.

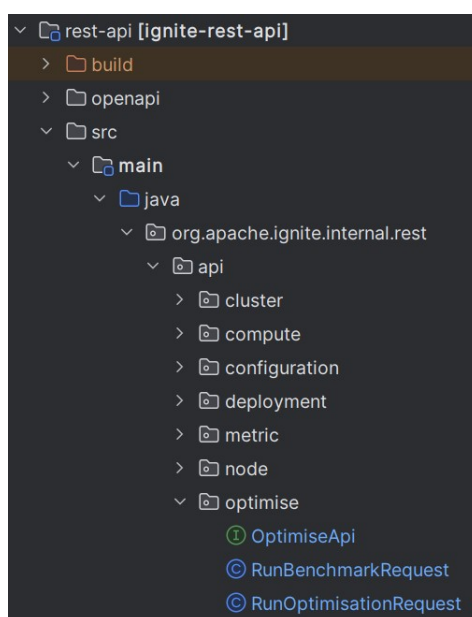


Рис.3.5. Содержимое модуля `rest-api`

Модуль `ignite-rest-api` содержит спецификации, используемые для автоматической генерации API посредством `openapi`.

Рассмотрим классы `RunBenchmarkRequest` и `RunOptimiseRequest`. Они включают в себя инструкции для сериализации и десериализации тела запросов (`runBenchmark` и `runOptimise` соответственно) используя формат JSON. Конструктор объекта помечен аннотацией `@JsonCreator`, который определяет как из json строки получить объект запроса. `@JsonProperty` позволяет считать параметр из строки и преобразовать в поле объекта. Геттеры помечены аннотацией `@JsonGetter`, и определяют как получить значения параметров при сериализации объекта в json формат.

Перейдем к интерфейсу `OptimiseApi`, определяющему методы для запуска бенчмарка, запуска оптимизации и получения результатов. `@JsonController` позволяет генератору спецификаций понять, что из данного интерфейса необходимо сгенерировать спецификацию и задает URL, по которому будет доступен данный контроллер. `@Post` и `@Get` задают методы как REST-запросы с соответствующим типом и содержат название методов. `@ApiResponse` задает ответы, которые может вернуть данный метод.

3.2.3. *ignite-rest*

Классы данного модуля также находится в общем пакете из-за несовершенства структуры проекта, содержимое модуля изображено на рисунке 3.6.

Класс `OptimiseController` является реализацией `OptimiseApi` и перенаправляет принятые запросы для дальнейшей обработки в класс `OptimiseManager`. `OptimiseFactory` служит для создания и менеджмента объекта `OptimiseManager`, а также для очистки ресурсов после использования.

3.2.4. *ignite-optimiser*

Логика оптимизатора была выделена в отдельный модуль `ignite-optimiser` и представлена на рисунке 3.7.

Координация действий выделена в класс `OptimiserManager`, реализующий интерфейс `IgniteComponent`. Это значит, что классу необходимо определить действия, которые будут производиться при запуске и остановке узла - в данном случае мы регистрируем обработчик сообщений. Теперь все сообщения, относящиеся к группе `OptimiserMessageGroup`, будут обрабатываться методом

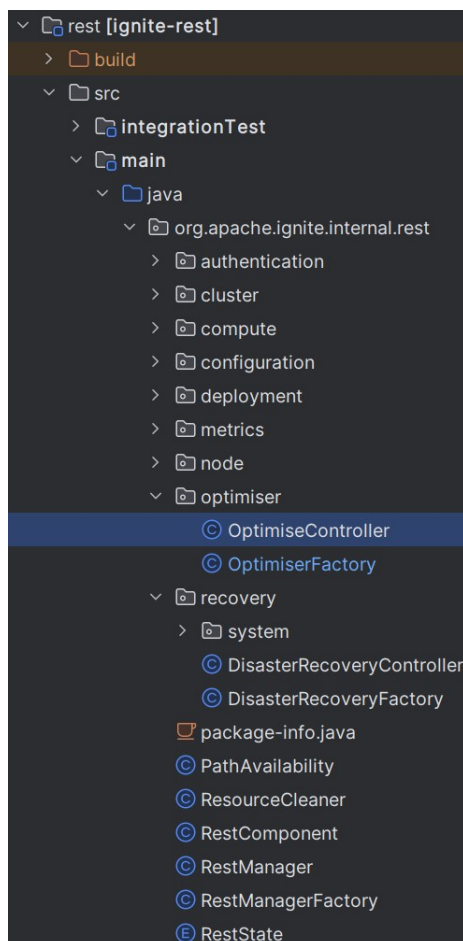


Рис.3.6. Содержимое модуля rest

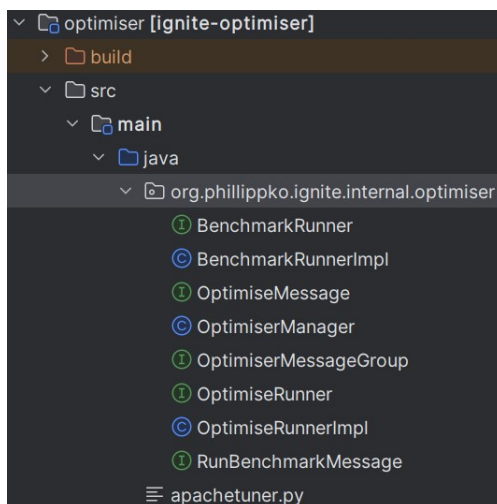


Рис.3.7. Содержимое модуля optimiser

handleMessage. Его логика проста - в зависимости от типа сообщения вызывается либо runBenchmarkInternal, либо runOptimiseInternal.

Внутри данных методов создается запись в MetastorageManager (хранилище метаданных, общее для всех узлов) со значением STARTED, дальше ответственность передается объектам BenchmarkRunner и OptimiseRunner, содержащим собственно логику оптимизации и запуска бенчмарков.

Следует заметить, что эти “раннеры” определены интерфейсами, а не конкретными реализациями, и передаются в конструкторе менеджера, а не создаются внутри. Это позволяет реализовать паттерн `Dependency Injection` и подменять реализации не трогая остальной код. Таким образом появляется возможность реализовывать альтернативные оптимизаторы, а также создавать “моки” для проведения юнит-тестирования.

Довольно интересна и логика `RunBenchmarkMessage` и `RunOptimiseMessage`. Они также являются интерфейсами. Это связано с тем, что `NetworkMessages` автоматически генерируется по заданным в этих интерфейсах правилам. Для каждого сообщения будет определена имплементация, фабрика, сериализатор и десериализатор, для унификации обработки всех сообщений.

Перейдем к классу `OptimiseRunnerImpl`. В нем определен один метод `getIssues`, внутри которого последовательно извлекаются разные значения из конфигурации (узла и кластера в целом) и в зависимости от заранее заданных эвристик определяются возможные проблемы. Обзор используемых эвристик проведен во второй главе. Кроме того, оптимизатор вызывает Python программу `arachetuner.py`, которая отвечает за запуск `OpenTuner`.

Благодаря использованию готового фреймворка, код `arachetuner.py` относительно прост. Есть метод для генерации конфигурации, в рамках “proof-of-concept” используется одна настройка, указывающая должна ли происходить синхронизация файлов RAFT с дисковым хранилищем. В текущей версии Apache Ignite данная настройка сильно влияет на производительность, и рекомендуется её отключать - в рамках тестирования будет проверено, выявит ли данную проблему `OpenTuner`.

Второй метод в рамках `arachetuner.py` отвечает за запуск бенчмарка и получение его результатов. Как было указано выше, в качестве примера был использован `UpsertKvBenchmark`. Код бенчмарка был переработан для записи результатов запуска в текстовый файл, который дальше считывает `OpenTuner` и делает выводы и скорости работы системы.

3.3. Выводы

Выбор технологий, использованных при разработке утилиты, в основном определён технологиями, уже использующимися проектом Apache Ignite. Применение готового фреймворка `OpenTuner` позволило использовать перспективные

технологии машинного обучения без сложностей, связанных с разработкой подобной программы с нуля. Разделение кода утилиты и Apache Ignite упрощает дальнейшую поддержку. Все сформулированные в предыдущей главе технические требования выполнены, и разработанный продукт позволяет решить поставленные задачи.

ГЛАВА 4. ТЕСТИРОВАНИЕ И РЕЗУЛЬТАТЫ АПРОБАЦИИ ОПТИМИЗАТОРА

4.1. Модульное тестирование

Разработанное приложение состоит из множества связанных между собой компонентов, и небольшие изменения в любом из них приводят к неочевидным проблемам в работе всей программы. Для решения данной проблемы код был покрыт модульными тестами, которые запускаются при любом изменении и позволяют обнаружить ошибки в момент их появления. Полный код тестов приведен в приложении 2.

Для разработки unit-тестов используется библиотека JUnit5 Jupiter. Использование данной библиотеки позволяет определить общие методы, вызываемые перед каждым или перед всеми тестовыми сценариями. Для того, чтобы изолировать каждый модуль, используется библиотека Mockito. Таким образом все смежные компоненты объявлены как “моки” - вместо вызова настоящего метода они возвращают заранее заданный результат, кроме того возможно отслеживать все взаимодействия с моком и выдавать ошибку если число вызовов методов или использованные аргументы не соответствуют ожиданиям.

Модульное тестирование проходит быстро, поэтому возможно протестировать множество сценариев за короткое время и запускать их в автоматическом режиме после каждого изменения кода.

Полный код тестов приведен в приложении 2.

4.2. Интеграционное тестирование

Модульные тесты не могут выявить проблемы на стыке разных компонентов, а также утомительны для написания и поддержки в актуальном состоянии – изменение логики класса требует переписывания тестов, а также изменения используемых моков. Даже простые компоненты могут взаимодействовать с десятком смежных сервисов и вызывать различные методы в различных вариациях аргументов.

Описанные выше недостатки модульного тестирования привели к необходимости написания более сложных и “дорогостоящих” тестов. В рамках интеграционного тестирования все сервисы создаются как и при настоящем запуске проекта, что может занимать довольно длительное время. Кроме того, при грамотной ин-

капсуляции (сокрытии внутренней логики) проекта далеко не всегда возможно создать полноценные тесты, проверяющие корректность всей логики. В связи с этим интеграционные тесты только проверяют возможность взаимодействия между сервисами и не проверяют корректность их работы - если взаимодействие прошло успешно и ошибки не получено, тест считается пройденным.

В случае тестирования командной строки происходит полный цикл работы проекта. Создается командная строка, создается кластер Apache Ignite из нескольких узлов. Вызывается команда и проверяется, что в ходе работы не произошло никаких ошибок и запрос действительно был создан. Тесты основаны на обширном фреймворке, созданном для тестирования всех компонентов Apache Ignite, что позволяет писать интеграционные тесты используя уже готовые “блоки”, не задумываясь о логике создания узлов, инициализации кластера и т.д.

Использование интеграционных тестов позволило убедиться в соблюдении контрактов взаимодействия между системами утилиты.

4.3. Функциональное тестирование

Функциональное тестирование является логичным продолжением интеграционного и модульного тестирования. В данном случае необходимо не только проверить корректность взаимодействия всех компонентов, но и удостовериться, что в ходе данного взаимодействия ещё и получены ожидаемые результаты. Автоматизация функциональных тестов является сложной задачей и часто в небольших проектах оказывается излишней. В связи с этим тестирование было проведено вручную, по заранее составленному списку тестовых сценариев. Подобное тестирование позволило избежать проблем при передаче получившейся программы на апробацию.

Пример такого сценария:

- Запустить два узла Apache Ignite.
- Запустить экземпляр командной строки.
- Инициализировать кластер из данных узлов.
- Подключиться к первому узлу.
- Обновить конфигурацию второго узла, добавив профиль с использованием системы “rocksdb”.

- Обновить конфигурацию второго узла, добавив профиль с использованием системы “aimem” и размером, превышающим выделенное под узел количество оперативной памяти.
- Запустить оптимизацию без флага `-write-intensive`, с указанием названия второго узла.
- Проверить логи второго узла, удостовериться что запрос был выполнен на переданном узле.
- Вызвать команду `result` с полученным ID операции.
- Удостовериться, что в результатах есть два предложения по оптимизации - использование `airpersist` вместо `rocksdb` и уменьшение выделенной под `volatile` профиль оперативной памяти

Благодаря относительно небольшому функционалу утилиты ручное тестирование всех вариантов использования производилось после каждого крупного изменения кода и не потребовало слишком больших трудозатрат, что позволило избежать более крупной задачи по автоматизации функционального тестирования.

4.4. Защита данных

Ограничение доступа и тестирование уязвимостей является важной частью разработки любого приложения. Тем не менее, внедрение утилиты в уже существующий продукт позволяет избежать данной необходимости - аутентификацией занимается отдельный модуль, и все команды утилиты автоматически можно считать защищенными - постольку, поскольку защищен кластер целиком.

4.5. Апробация

Апробация проводилась для оценки эффективности и потенциала применения разработанной утилиты. Проведенное тестирование с участием трех добровольцев позволило проверить гипотезу, что с использованием системы пользователь без опыта конфигурации Apache Ignite сможет за короткое время добиться результатов, сходных с результатами опытного администратора или неопытного пользователя, посвятившего заметно большее количество времени данной задаче.

Апробация производилась на ноутбуке с 16 гигабайтами DDR5 ОЗУ, процессором i9-13900H (14 ядер, 20 потоков, пиковая частота 5.4GHz) и SSD диском

(последовательная запись 4419 MB/c, последовательное чтение 3094 MB/c, случайное чтение 2402 MB/c).

Была подготовлена тестовая конфигурация. Под JVM выделено 1024MB памяти, Под сборку мусора определено 4 потока. Время жизни данных выставлено в 16 минут (1 млн миллисекунд). В качестве стандартной системы хранения выбран aipersist (основан на B+ деревьях и неоптимален для write-intensive приложений).

Проведём тестирование оптимизатора на исходной конфигурации кластера. На рисунке 4.1 приведен результат запуска оптимизатора.

```
[defaultNode]> optimise runOptimise --write-intensive
Optimisation was started successfully with id fdc99c12-1ece-4148-aa38-45004bf40f39
[defaultNode]> optimise result fdc99c12-1ece-4148-aa38-45004bf40f39
Optimisation result: More than 90% of maximum memory is used, increase -Xmx JVM option; Less than half
of available threads designated for GC and memory usage is high, increase gcConfiguration.threads, cu
rrent: DynamicProperty [readOnly=false, injectedNameField=false, internalIdField=false, key=threads, v
alue=20]; Low watermark data availability is too high, causing extra disk usage: DynamicProperty [read
Only=false, injectedNameField=false, internalIdField=false, key=dataAvailabilityTime, value=600000]; L
ow watermark update interval is too high [updateInterval=DynamicProperty [readOnly=false, injectedName
Field=false, internalIdField=false, key=updateInterval, value=300000], dataAvailabilityTime=DynamicPro
perty [readOnly=false, injectedNameField=false, internalIdField=false, key=dataAvailabilityTime, value
=600000]]; Volatile data regions size OK; For write-intensive workloads RocksDB should be used instead
page memory in these profiles: default; Not running OpenTuner
```

Рис.4.1. Запуск оптимизатора на исходной конфигурации

На рисунке 4.2 приведены результаты запуска бенчмарка, вставляющего 30 тысяч значений в пустую таблицу, на исходной конфигурации.

```
Do you want to reconnect to the last connected node http://localhost:10300? [Y/n] y
Connected to http://localhost:10300
The cluster is not initialized. Run cluster init command to initialize it.
[defaultNode]> cluster init --name name --metastorage-group defaultNode
Cluster was initialized successfully
[defaultNode]> optimise runBenchmark --values 30000 --iterations 8
Benchmark was started successfully with id d120a679-1d89-4e20-b961-ad2d821d420b
[defaultNode]> optimise result d120a679-1d89-4e20-b961-ad2d821d420b
Optimisation result: Iteration 1 finished in 16022MS
Iteration 2 finished in 10898MS
Iteration 3 finished in 10644MS
Iteration 4 finished in 10506MS
Iteration 5 finished in 10957MS
Iteration 6 finished in 11031MS
Iteration 7 finished in 11267MS
Iteration 8 finished in 11402MS
```

Рис.4.2. Запуск бенчмарка на исходной конфигурации

Применим рекомендации и проведем тестирование на получившейся конфигурации. На рисунке 4.3 приведен результат запуска оптимизатора после исправлений.

```
[defaultNode]> optimise runOptimise --write-intensive --tuner-timeout 10000
Optimisation was started successfully with id e71ed01d-4591-44ef-b3d2-8618185dabdb
[defaultNode]> optimise result e71ed01d-4591-44ef-b3d2-8618185dabdb
Optimisation result: Maximum memory OK; GC threads OK; Low watermark data availability OK; Low watermark update interval
OK; Volatile data regions size OK; For write-intensive workloads RocksDB should be used instead page memory in these pr
ofiles: default; OpenTuner recommended values using UpsertKvBenchmark for %s milliseconds: [fsync=false]
[defaultNode]>
```

Рис.4.3. Запуск оптимизатора на итоговой конфигурации

На рисунке 4.4 приведён результат запуска бенчмарка после исправлений.

```
[defaultNode]> cluster init --name name --metastorage-group defaultNode
Cluster was initialized successfully
[defaultNode]> optimise runBenchmark --values 30000 --iterations 8 --profile rocksdb_profile
Benchmark was started successfully with id 9f0c7549-118f-4752-8220-718dfc91adb6
[defaultNode]> optimise result 9f0c7549-118f-4752-8220-718dfc91adb6
Optimisation result: Benchmark STARTED
[defaultNode]> optimise result 9f0c7549-118f-4752-8220-718dfc91adb6
Optimisation result: Iteration 1 finished in 9036MS
Iteration 2 finished in 7523MS
Iteration 3 finished in 7982MS
Iteration 4 finished in 7728MS
Iteration 5 finished in 8322MS
Iteration 6 finished in 8242MS
Iteration 7 finished in 8305MS
Iteration 8 finished in 8801MS
```

Рис.4.4. Запуск бенчмарка на итоговой конфигурации

По результатам тестирования сделан вывод, что оптимизатор выявляет все ожидаемые проблемы конфигурации, позволяет увеличить производительность кластера, и не выводит проблемы для конфигурации, соответствующей рекомендациям.

В качестве добровольцев для дальнейшего испытания оптимизатора вызвались два студента (С1 и С2) без прошлого опыта конфигурации Apache Ignite и опытный разработчик (ОР) из команды Apache Ignite. Всем троим была выдана подготовленная конфигурация, ссылка на документацию Apache Ignite и доступ к тестовому кластеру. Второй студент (С2) получил версию кластера со встроенным оптимизатором. Задача - в течение 30 минут настроить кластер для использования write-intensive приложением.

Все три добровольца исправили использование `airpersist` вместо `RocksDB`, частоту обновления `LowWatermark` и число потоков под сборку мусора (данные рекомендации содержатся в разных частях документации). ОР кроме того исправил низкое значение выделенной под JVM памяти. С2 исправил все выявляемые эвристиками проблемы конфигурации и получившаяся конфигурация соответствует итоговой конфигурации, использовавшейся при первичном тестировании.

С1 потратил на конфигурацию все 30 минут, внимательно изучая все страницы конфигурации, в поиске советов по оптимизации. С2 потребовалось около 10 минут - в основном они ушли на то, чтобы разобраться как применить полученные в ходе выполнения оптимизатора рекомендации. ОР закончил в течение 3 минут, зная куда смотреть, и на что обратить внимание.

По итогам тестирования итоговых конфигураций была составлена таблица 4.1, время запусков измеряется в секундах. Для конфигураций подсчитаны среднее, медиана и среднее квадратичное отклонение.

Из таблицы видно, что все три участника повысили производительность кластера - в основном благодаря переходу с `airpersist` на `RocksDB`. Первый за-

Таблица 4.1

Результаты тестирования конфигураций

Итерация	Исходная	C1	OP	C2
1	16.022	8.985	8.994	9.036
2	10.898	7.631	7.652	7.523
3	10.644	8.032	7.897	7.982
4	10.506	8.127	7.931	7.728
5	10.957	8.456	8.256	8.322
6	11.031	8.587	8.355	8.242
7	11.267	8.811	8.297	8.305
8	11.402	9.188	8.681	8.801
Среднее	11.5909	8.4771	8.2579	8.2424
СКО	1.6972	0.4904	0.4087	0.4746

пуск происходит медленнее всех последующих, возможно система производит дополнительные действия при вставке данных в только что созданную таблицу. Из-за того, что C1 упустил низкое значение выделенной под JVM оперативной памяти, последующие запуски замедлялись быстрее, чем в конфигурациях C2 и OP. Таким образом, применение рекомендаций системы позволило увеличить среднюю производительность на 40 процентов при вставке 30 тысяч значений в течение 8 итераций.

4.6. Дальнейшее развитие

Потенциал утилиты ограничен вследствие отсутствия базовых инструментов администрирования в данной версии Apache Ignite. Одним из перспективных вариантов увеличения эффективности оптимизации может стать анализ запросов к базе данных, с целью выявления недостающих индексов и переработки логической структуры базы. Кроме того, автоматический мониторинг метрик при настоящей нагрузке на базу поможет более эффективно выявлять проблемы в конфигурации системы вместо ручного запуска синтетических бенчмарков. Не менее важно, что в ходе разработки было допущено множество упрощений, вызванных масштабом поставленной задачи. Исправление данных проблем также позволит повысить эффективность утилиты и пользовательский опыт при её использовании.

4.7. Выводы

Все выявленные в ходе тестирования проблемы были исправлены, наличие автоматических тестов позволяет вносить дальнейшие изменения в код проекта и отлавливать некоторые баги без ручного запуска системы. В ходе апробации выявлено, что разработанный инструмент позволяет неопытному пользователю выполнять конфигурацию кластера заметно быстрее чем просто читая документацию, приблизиться к результатам опытного администратора, и выявлять проблемы, неочевидные при простом чтении текстовой конфигурации.

ЗАКЛЮЧЕНИЕ

В ходе выполнения данной работы была разработана утилита для автоматической оптимизации конфигурации кластера распределенной базы данных Apache Ignite. Проведенное исследование подтвердило актуальность темы, обусловленную сложностью настройки СУБД и отсутствием готовых инструментов для конфигурирования Apache Ignite.

В рамках исследования были решены следующие задачи:

- Проведен анализ существующих решений для оптимизации баз данных и выявлены их недостатки в контексте Apache Ignite.
- Разработана утилита, интегрированная в командную строку Apache Ignite, обеспечивающая автоматическую оценку конфигурации и подбор оптимальных параметров.
- Реализованы два подхода к оптимизации: эвристический анализ конфигурации и автоматизированный подбор параметров с использованием фреймворка OpenTuner.
- Проведено тестирование разработанной утилиты, продемонстрировавшее её эффективность.

Результаты апробации показали, что предложенная утилита позволяет значительно упростить процесс настройки Apache Ignite, особенно для пользователей без глубоких знаний о внутреннем устройстве системы. По сравнению с ручной настройкой утилита помогла неопытным пользователям быстрее выявлять ошибки конфигурации и принимать обоснованные решения для их исправления.

Практическое значение работы заключается в создании инструмента, который может быть использован администраторами баз данных и разработчиками для упрощения настройки кластеров Apache Ignite. Учитывая открытость кода, возможна дальнейшая интеграция утилиты в основную кодовую базу Apache Ignite или её адаптация для работы с другими распределенными СУБД.

В перспективе возможно расширение функционала утилиты путем интеграции механизма анализа запросов, автоматизированного мониторинга метрик работы базы данных, а также разработки графического интерфейса для удобства пользователей.

Разработанная утилита доказала свою эффективность и может быть рекомендована для дальнейшего использования и развития в сообществе пользователей Apache Ignite.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. AI-powered database tuning. — URL: https://app.dbtune.com/DBtune_Product_paper.pdf (дата обращения: 28.01.2025).
2. *Ansel J.* OpenTuner Github. — URL: <https://github.com/jansel/opentuner> (дата обращения: 28.01.2025).
3. *Ansel J., Kamil S., Veeramachaneni K.* OpenTuner: An Extensible Framework for Program Autotuning // Proceedings of the 23rd International Conference on Parallel Architectures and Compilation. — Edmonton, AB, Canada: Association for Computing Machinery, 2014. — С. 303—316. — (Сер.: PACT '14). — DOI 10.1145/2628071.2628092. — URL: <https://doi.org/10.1145/2628071.2628092>.
4. Apache Ignite Documentation. — URL: <https://ignite.apache.org/docs/latest/> (дата обращения: 28.01.2025).
5. *Bosboom J., Rajadurai S., Wong W.* StreamJIT: A Commensal Compiler for High-performance Stream Programming // SIGPLAN Not. — New York, NY, USA, 2014. — Т. 49, № 10. — С. 177—195. — DOI 10.1145/2714064.2660236. — URL: <https://doi.org/10.1145/2714064.2660236>.
6. *Chen C.* CDBTune Github. — URL: <https://github.com/HustAIsGroup/CDBTune/> (дата обращения: 28.01.2025).
7. Db2 Design Advisor. — URL: <https://www.ibm.com/docs/en/db2/11.5?topic=commands-db2advis-db2-design-advisor/> (дата обращения: 28.01.2025).
8. DBEngines Ranking. — URL: <https://db-engines.com/en/ranking> (дата обращения: 28.01.2025).
9. *Dias K., Ramacher M., Shaft U.* Automatic Performance Diagnosis and Tuning in Oracle // CIDR. — 2005. — № 7. — С. 328—334.
10. Featured Customers - GridGain Systems. — URL: <https://www.gridgain.com/solutions/customers> (дата обращения: 28.01.2025).
11. Ottertune. — URL: <https://ottertune.com/> (дата обращения: 28.01.2025).
12. Platform V DataGrid. — URL: <https://platformv.sbertech.ru/products/rabota-s-dannymi/datagrid> (дата обращения: 28.01.2025).
13. *Skellely A.* An Optimizer Smart Enough to Recommend Its Own Indexes // Proceedings of the 16th International Conference on Data Engineering. — USA: IEEE Computer Society, 2000. — С. 101. — (Сер.: ICDE '00).
14. *Smith G.* PGTune Github. — URL: <https://github.com/gregs1104/pgtune/> (дата обращения: 28.01.2025).

15. *Van Aken D., Pavlo A., Gordon.* Automatic Database Management System Tuning Through Large-scale Machine Learning // Proceedings of the 2017 ACM International Conference on Management of Data. — Chicago, Illinois, USA: Association for Computing Machinery, 2017. — С. 1009—1024. — (Сер.: SIGMOD '17). — DOI 10.1145/3035918.3064029. — URL: <https://doi.org/10.1145/3035918.3064029>.

16. *Zhang J., Liu Y., Zhou K.* An End-to-End Automatic Cloud Database Tuning System Using Deep Reinforcement Learning. — 2019. — DOI 10.1145/3299869.3300085. — URL: <https://doi.org/10.1145/3299869.3300085>.

17. *Zhou X.* Researches and Practices in Autonomous Databases. — URL: <https://github.com/TsinghuaDatabaseGroup/AIDB> (дата обращения: 28.01.2025).

18. СберТех и Digital Design помогут бизнесу перейти на российское ПО. — URL: <https://platformv.sbertech.ru/blog/sber-teh-i-digital-design-pomogut-biznesu-perejti-na-rossijskoe-po> (дата обращения: 28.01.2025).

Приложение 1

Исходный код приложения

```

package org.phillippko.ignite.internal.cli.optimise.call;

import static org.apache.ignite.internal.cli.core.call.
    DefaultCallOutput.failure;
5 import static org.apache.ignite.internal.cli.core.call.
    DefaultCallOutput.success;

import jakarta.inject.Singleton;
import org.apache.ignite.internal.cli.core.call.Call;
import org.apache.ignite.internal.cli.core.call.
    DefaultCallOutput;
10 import org.apache.ignite.internal.cli.core.exception.
    IgniteCliApiException;
import org.apache.ignite.internal.cli.core.rest.
    ApiClientFactory;
import org.apache.ignite.rest.client.api.OptimiseApi;
import org.apache.ignite.rest.client.invoker.ApiException;

15 @Singleton
public class GetResultCall implements Call<GetResultCallInput,
    String> {
    private final ApiClientFactory clientFactory;

    public GetResultCall(ApiClientFactory clientFactory) {
20     this.clientFactory = clientFactory;
    }

    @Override
    public DefaultCallOutput<String> execute(GetResultCallInput
        input) {
25     try {
        OptimiseApi optimiseApi = new OptimiseApi(clientFactory.
            getClient(input.getClusterUrl()));

        return success(optimiseApi.optimisationResult(input.
            getId()));
    } catch (ApiException e) {
30     return failure(new IgniteCliApiException(e, input.
        getClusterUrl()));
    }
}

```

```

    }
}

package org.phillippko.ignite.internal.cli.optimise.call;

import java.util.UUID;
import org.apache.ignite.internal.cli.core.call.CallInput;

public class GetResultCallInput implements CallInput {
    private final String clusterUrl;

    private final UUID id;

    public GetResultCallInput(String clusterUrl, UUID id) {
        this.clusterUrl = clusterUrl;
        this.id = id;
    }

    public static GetResultInputBuilder builder() {
        return new GetResultInputBuilder();
    }

    public String getClusterUrl() {
        return clusterUrl;
    }

    public UUID getId() {
        return id;
    }

    public static class GetResultInputBuilder {
        private String clusterUrl;
        private UUID id;

        public GetResultInputBuilder setClusterUrl(String
            clusterUrl) {
            this.clusterUrl = clusterUrl;

            return this;
        }

        public GetResultInputBuilder setId(UUID id) {
            this.id = id;

            return this;
        }
    }
}

```

```

        }

45     public GetResultCallInput build() {
        return new GetResultCallInput(clusterUrl, id);
    }
}

package org.phillippko.ignite.internal.cli.optimise.call;

import static org.apache.ignite.internal.cli.core.call.
    DefaultCallOutput.failure;
5 import static org.apache.ignite.internal.cli.core.call.
    DefaultCallOutput.success;

import jakarta.inject.Singleton;
import org.apache.ignite.internal.cli.core.call.Call;
import org.apache.ignite.internal.cli.core.call.
    DefaultCallOutput;
10 import org.apache.ignite.internal.cli.core.exception.
    IgniteCliApiException;
import org.apache.ignite.internal.cli.core.rest.
    ApiClientFactory;
import org.apache.ignite.rest.client.api.OptimiseApi;
import org.apache.ignite.rest.client.invoker.ApiException;
import org.apache.ignite.rest.client.model.RunBenchmarkRequest
    ;
15
@Singleton
public class RunBenchmarkCall implements Call<
    RunBenchmarkCallInput, String> {
    private final ApiClientFactory clientFactory;

20     public RunBenchmarkCall(ApiClientFactory clientFactory) {
        this.clientFactory = clientFactory;
    }

    @Override
25     public DefaultCallOutput<String> execute(
        RunBenchmarkCallInput input) {
        try {
            OptimiseApi optimiseApi = new OptimiseApi(clientFactory.
                getClient(input.getClusterUrl()));
            RunBenchmarkRequest request = new RunBenchmarkRequest()
                .benchmarkFilePath(input.getBenchmarkFilePath())

```

```

30     .nodeName(input.getNodeName())
        .iterations(input.getIterations())
        .profile(input.getProfile())
        .values(input.getValues());

35     return success("Benchmark was started successfully with
        id " + optimiseApi.runBenchmark(request));
    } catch (ApiException e) {
        return failure(new IgniteCliApiException(e, input.
            getClusterUrl()));
    }
}
40 }

package org.phillippko.ignite.internal.cli.optimise.call;

import static org.apache.ignite.internal.cli.core.call.
    DefaultCallOutput.failure;
5 import static org.apache.ignite.internal.cli.core.call.
    DefaultCallOutput.success;

import jakarta.inject.Singleton;
import org.apache.ignite.internal.cli.core.call.Call;
import org.apache.ignite.internal.cli.core.call.
    DefaultCallOutput;
10 import org.apache.ignite.internal.cli.core.exception.
    IgniteCliApiException;
import org.apache.ignite.internal.cli.core.rest.
    ApiClientFactory;
import org.apache.ignite.rest.client.api.OptimiseApi;
import org.apache.ignite.rest.client.invoker.ApiException;
import org.apache.ignite.rest.client.model.
    RunBenchmarkRequest;
15

@Singleton
public class RunBenchmarkCall implements Call<
    RunBenchmarkCallInput, String> {
    private final ApiClientFactory clientFactory;

20     public RunBenchmarkCall(ApiClientFactory clientFactory) {
        this.clientFactory = clientFactory;
    }

    @Override

```

```

25 public DefaultCallOutput<String> execute(
    RunBenchmarkCallInput input) {
    try {
        OptimiseApi optimiseApi = new OptimiseApi(
            clientFactory.getClient(input.getClusterUrl()));
        RunBenchmarkRequest request = new RunBenchmarkRequest
            ()
            .benchmarkFilePath(input.getBenchmarkFilePath())
30         .nodeName(input.getNodeName())
            .iterations(input.getIterations())
            .profile(input.getProfile())
            .values(input.getValues());

        return success("Benchmark was started successfully
            with id " + optimiseApi.runBenchmark(request));
    } catch (ApiException e) {
        return failure(new IgniteCliApiException(e, input.
            getClusterUrl()));
    }
    }
40 }

package org.phillippko.ignite.internal.cli.optimise.call;

import javax.annotation.Nullable;
5 import org.apache.ignite.internal.cli.core.call.CallInput;

public class RunBenchmarkCallInput implements CallInput {
    private final String clusterUrl;

    private final String benchmarkFilePath;
    private final String nodeName;
    private final int iterations;
    private final @Nullable Long values;
    private final String profile;
15

    public RunBenchmarkCallInput(
        String clusterUrl,
        @Nullable String nodeName,
        @Nullable String benchmarkFilePath,
20         int iterations,
        @Nullable Long values,
        String profile
    ) {
        this.clusterUrl = clusterUrl;

```

```

25         this.benchmarkFilePath = benchmarkFilePath;
           this.nodeName = nodeName;
           this.iterations = iterations;
           this.values = values;
           this.profile = profile;
30     }

           public static RunBenchmarkCallInputBuilder builder() {
               return new RunBenchmarkCallInputBuilder();
           }

35     public String getClusterUrl() {
           return clusterUrl;
       }

40     public @Nullable String getBenchmarkFilePath() {
           return benchmarkFilePath;
       }

           public @Nullable String getNodeName() {
45         return nodeName;
       }

           public int getIterations() {
50         return iterations;
       }

           public @Nullable Long getValues() {
               return values;
       }

55     public String getProfile() {
           return profile;
       }

60     public static class RunBenchmarkCallInputBuilder {
           private String clusterUrl;
           private @Nullable String benchmarkFilePath;
           private @Nullable String nodeName;
           private int iterations;
65         private @Nullable Long values;
           private String profile;

           public RunBenchmarkCallInputBuilder setClusterUrl(String
               clusterUrl) {

```

```
70         this.clusterUrl = clusterUrl;

        return this;
    }

    public RunBenchmarkCallInputBuilder setBenchmarkFilePath
75         (String benchmarkFilePath) {
        this.benchmarkFilePath = benchmarkFilePath;

        return this;
    }

80    public RunBenchmarkCallInputBuilder setNodeName(String
        nodeName) {
        this.nodeName = nodeName;

        return this;
    }

85    public RunBenchmarkCallInputBuilder setIterations(int
        iterations) {
        this.iterations = iterations;

        return this;
90    }

    public RunBenchmarkCallInputBuilder setValues(@Nullable
        Long values) {
        this.values = values;

95        return this;
    }

    public RunBenchmarkCallInputBuilder setProfile(String
        profile) {
        this.profile = profile;

100        return this;
    }

    public RunBenchmarkCallInput build() {
105        return new RunBenchmarkCallInput(clusterUrl, nodeName,
            benchmarkFilePath, iterations, values, profile);
    }
}
```

```

    }

    package org.phillippko.ignite.internal.cli.optimise.call;

    import static org.apache.ignite.internal.cli.core.call.
        DefaultCallOutput.failure;
5    import static org.apache.ignite.internal.cli.core.call.
        DefaultCallOutput.success;

    import jakarta.inject.Singleton;
    import org.apache.ignite.internal.cli.core.call.Call;
    import org.apache.ignite.internal.cli.core.call.
        DefaultCallOutput;
10    import org.apache.ignite.internal.cli.core.exception.
        IgniteCliApiException;
    import org.apache.ignite.internal.cli.core.rest.
        ApiClientFactory;
    import org.apache.ignite.rest.client.api.OptimiseApi;
    import org.apache.ignite.rest.client.invoker.ApiException;
    import org.apache.ignite.rest.client.model.
        RunOptimisationRequest;
15

    @Singleton
    public class RunOptimiseCall implements Call<
        RunOptimiseCallInput, String> {
        private final ApiClientFactory clientFactory;

20        public RunOptimiseCall(ApiClientFactory clientFactory) {
            this.clientFactory = clientFactory;
        }

        @Override
25        public DefaultCallOutput<String> execute(
            RunOptimiseCallInput input) {
            try {
                OptimiseApi optimiseApi = new OptimiseApi(
                    clientFactory.getClient(input.getClusterUrl()));

                RunOptimisationRequest request = new
                    RunOptimisationRequest()
30                .writeIntensive(input.getWriteIntensive())
                .tunerTimeout(input.getTunerTimeout())
                .nodeName(input.getNodeName());

```

```

        return success("Optimisation was started successfully
                        with id " + optimiseApi.optimise(request));
35    } catch (ApiException e) {
        return failure(new IgniteCliApiException(e, input.
            getClusterUrl()));
    }
}
}

package org.phillippko.ignite.internal.cli.optimise.call;

import org.apache.ignite.internal.cli.core.call.CallInput;
5
public class RunOptimiseCallInput implements CallInput {
    private final String clusterUrl;
    private final String nodeName;
    private final boolean writeIntensive;
10    private final int tunerTimeout;

    public RunOptimiseCallInput(String clusterUrl, String
        nodeName, boolean writeIntensive, int tunerTimeout) {
        this.clusterUrl = clusterUrl;
        this.nodeName = nodeName;
15        this.writeIntensive = writeIntensive;
        this.tunerTimeout = tunerTimeout;
    }

    public static RunOptimiseCallInputBuilder builder() {
20        return new RunOptimiseCallInputBuilder();
    }

    public String getClusterUrl() {
25        return clusterUrl;
    }

    public int getTunerTimeout() {
        return tunerTimeout;
    }
30

    public boolean getWriteIntensive() {
        return writeIntensive;
    }

    public String getNodeName() {
35        return nodeName;
    }

```

```

    }

    public static class RunOptimiseCallInputBuilder {
40      private String clusterUrl;
        private String nodeName;
        private boolean writeIntensive;
        private int tunerTimeout;

45      public RunOptimiseCallInputBuilder setClusterUrl(String
          clusterUrl) {
          this.clusterUrl = clusterUrl;

          return this;
        }

50      public RunOptimiseCallInputBuilder setNodeName(String
          nodeName) {
          this.nodeName = nodeName;

          return this;
55      }

      public RunOptimiseCallInputBuilder setWriteIntensive(
          boolean writeIntensive) {
          this.writeIntensive = writeIntensive;

60      return this;
        }

      public RunOptimiseCallInputBuilder setTunerTimeout(int
          tunerTimeout) {
          this.tunerTimeout = tunerTimeout;

65      return this;
        }

      public RunOptimiseCallInput build() {
70      return new RunOptimiseCallInput(clusterUrl, nodeName,
          writeIntensive, tunerTimeout);
        }
    }
}

package org.phillippko.ignite.internal.cli.optimise.command;

```

```

import static org.apache.ignite.internal.cli.commands.
    Options.Constants.ID_OPTION_DESC;

import jakarta.inject.Inject;
import java.util.UUID;
import java.util.concurrent.Callable;
import org.apache.ignite.internal.cli.commands.BaseCommand;
import org.apache.ignite.internal.cli.commands.cluster.
    ClusterUrlMixin;
import org.apache.ignite.internal.cli.core.call.
    CallExecutionPipeline;
import org.phillippko.ignite.internal.cli.optimise.call.
    GetResultCall;
import org.phillippko.ignite.internal.cli.optimise.call.
    GetResultCallInput;
import picocli.CommandLine.Command;
import picocli.CommandLine.Mixin;
import picocli.CommandLine.Parameters;

@Command(name = "result", description = "Prints results of
    the optimisation or benchmark.")
public class GetResultCommand extends BaseCommand implements
    Callable<Integer> {
    @Mixin
    private ClusterUrlMixin clusterUrlMixin;

    @Inject
    private GetResultCall call;

    @Parameters(index = "0", description = ID_OPTION_DESC)
    private UUID id;

    @Override
    public Integer call() {
        return runPipeline(CallExecutionPipeline.builder(call)
            .inputProvider(this::buildCallInput)
            .output(spec.commandLine().getOut())
        );
    }

    private GetResultCallInput buildCallInput() {
        return GetResultCallInput.builder()
            .setClusterUrl(clusterUrlMixin.getClusterUrl())
            .setId(id)
            .build();
    }

```

```

    }
}

package org.phillippko.ignite.internal.cli.optimise.command;

import static org.apache.ignite.internal.cli.commands.
    Options.Constants.ID_OPTION_DESC;

5
import jakarta.inject.Inject;
import java.util.UUID;
import java.util.concurrent.Callable;
import org.apache.ignite.internal.cli.call.recovery.restart.
    RestartPartitionsCall;
10
import org.apache.ignite.internal.cli.call.recovery.restart.
    RestartPartitionsCallInput;
import org.apache.ignite.internal.cli.commands.BaseCommand;
import org.apache.ignite.internal.cli.commands.cluster.
    ClusterUrlMixin;
import org.apache.ignite.internal.cli.commands.questions.
    ConnectToClusterQuestion;
import org.apache.ignite.internal.cli.core.call.
    CallExecutionPipeline;
15
import org.apache.ignite.internal.cli.core.exception.handler
    .ClusterNotInitializedExceptionHandler;
import org.apache.ignite.internal.cli.core.flow.builder.
    Flows;
import org.phillippko.ignite.internal.cli.optimise.call.
    GetResultCall;
import org.phillippko.ignite.internal.cli.optimise.call.
    GetResultCallInput;
import picocli.CommandLine.Command;
20
import picocli.CommandLine.Mixin;
import picocli.CommandLine.Parameters;

@Command(name = "result", description = "Prints results of
    the optimisation or benchmark.")
public class GetResultReplCommand extends BaseCommand
    implements Runnable {
25
    @Inject
    private GetResultCall call;

    @Mixin
    private ClusterUrlMixin clusterUrl;

30
    @Parameters(index = "0", description = ID_OPTION_DESC)

```

```

private UUID id;

@Inject
35 private ConnectToClusterQuestion question;

@Override
public void run() {
    runFlow(question.askQuestionIfNotConnected(clusterUrl.
        getClusterUrl()))
40 .map(this::buildCallInput)
    .then(Flows.fromCall(call))
    .exceptionHandler(ClusterNotInitializedExceptionHandler.
        createReplHandler("Cannot get results"))
    .print()
    );
45 }

private GetResultCallInput buildCallInput(String url) {
    return GetResultCallInput.builder()
        .setClusterUrl(url)
50 .setId(id)
        .build();
    }
}

package org.phillippko.ignite.internal.cli.optimise.command;

import static org.apache.ignite.internal.cli.commands.
    Options.Constants.NODE_NAME_OPTION;
5 import static org.apache.ignite.internal.cli.commands.
    Options.Constants.NODE_NAME_OPTION_DESC;
import static org.apache.ignite.internal.cli.commands.
    Options.Constants.NODE_NAME_OPTION_SHORT;

import picocli.CommandLine.Option;

10 public class NodeNameMixin {
    @Option(names = {NODE_NAME_OPTION, NODE_NAME_OPTION_SHORT
        }, description = NODE_NAME_OPTION_DESC)
    private String nodeName;

    public String nodeName() {
15     return nodeName;
    }
}

```

```

package org.phillippko.ignite.internal.cli.optimise.command;

import org.apache.ignite.internal.cli.commands.BaseCommand;
import picocli.CommandLine.Command;

@Command(name = "optimise",
subcommands = {
    RunOptimiseCommand.class,
    RunBenchmarkCommand.class,
    GetResultCommand.class
},
description = "Tools for optimising cluster configuration.")
public class OptimiseCommand extends BaseCommand {

package org.phillippko.ignite.internal.cli.optimise.command;

import static org.apache.ignite.internal.cli.commands.
    Options.Constants.TUNER_TIMEOUT_OPTION;
import static org.apache.ignite.internal.cli.commands.
    Options.Constants.WRITE_INTENSIVE_OPTION;

import picocli.CommandLine.Option;

public class OptimiseMixin {
    @Option(names = WRITE_INTENSIVE_OPTION, description = "If
        target configuration should be prepared for write-
        intensive use-cases")
    private boolean writeIntensive;

    @Option(names = TUNER_TIMEOUT_OPTION, description = "How
        long should OpenTuner run in milliseconds",
        defaultValue = "0")
    private int tunerTimeout;

    public boolean writeIntensive() {
        return writeIntensive;
    }

    public int tunerTimeout() {
        return tunerTimeout;
    }
}

package org.phillippko.ignite.internal.cli.optimise.command;

```

```

import org.apache.ignite.internal.cli.commands.BaseCommand;
5 import org.apache.ignite.internal.cli.commands.recovery.
    partitions.reset.ResetPartitionsReplCommand;
import org.apache.ignite.internal.cli.commands.recovery.
    partitions.restart.RestartPartitionsReplCommand;
import org.apache.ignite.internal.cli.commands.recovery.
    partitions.states.PartitionStatesReplCommand;
import picocli.CommandLine.Command;

10 @Command(name = "optimise",
    subcommands = {
        GetResultReplCommand.class,
        RunOptimiseReplCommand.class,
        RunBenchmarkReplCommand.class
15 },
    description = "Tools for optimising cluster configuration.")
public class OptimiseReplCommand extends BaseCommand {
}

package org.phillippko.ignite.internal.cli.optimise.command;

import static org.apache.ignite.internal.cli.commands.
    Options.Constants.NODE_NAME_OPTION;
5 import static org.apache.ignite.internal.cli.commands.
    Options.Constants.NODE_NAME_OPTION_DESC;
import static org.apache.ignite.internal.cli.commands.
    Options.Constants.NODE_NAME_OPTION_SHORT;

import jakarta.inject.Inject;
import java.util.concurrent.Callable;
10 import org.phillippko.ignite.internal.cli.optimise.call.
    RunBenchmarkCall;
import org.phillippko.ignite.internal.cli.optimise.call.
    RunBenchmarkCallInput;
import org.apache.ignite.internal.cli.commands.BaseCommand;
import org.apache.ignite.internal.cli.commands.cluster.
    ClusterUrlMixin;
import org.apache.ignite.internal.cli.core.call.
    CallExecutionPipeline;
15 import picocli.CommandLine.Command;
import picocli.CommandLine.Mixin;
import picocli.CommandLine.Option;
import picocli.CommandLine.Parameters;

```

```

20  @Command(name = "runBenchmark", description = "Runs
    benchmark")
public class RunBenchmarkCommand extends BaseCommand
    implements Callable<Integer> {
    /** Node URL option. */
    @Mixin
    private ClusterUrlMixin clusterUrlMixin;

25
    @Inject
    private RunBenchmarkCall call;

    @Parameters(
30    arity = "0..1",
    index = "0",
    description = "Path to the file containing SQL queries to
        run during benchmark."
    )
    private String benchmarkFilePath;

35
    @Mixin
    private NodeNameMixin nodeName;

    @Option(names = "--iterations", description = "Number of
        iterations to run the benchmark.", defaultValue = "1")
40    private int iterations;

    @Option(names = "--values", description = "Number of
        values to run the benchmark.")
    private Long values;

45
    @Option(names = "--profile", description = "Storage
        profile to use for the benchmark.", defaultValue = "
        default")
    private String profile;

    @Override
    public Integer call() {
50        return runPipeline(CallExecutionPipeline.builder(call)
            .inputProvider(this::buildCallInput)
            .output(spec.commandLine().getOut())
            );
    }

55
    private RunBenchmarkCallInput buildCallInput() {
        return RunBenchmarkCallInput.builder()

```

```

        .setClusterUrl(clusterUrlMixin.getClusterUrl())
        .setBenchmarkFilePath(benchmarkFilePath)
60    .setNodeName(nodeName.nodeName())
        .setIterations(iterations)
        .setValues(values)
        .setProfile(profile)
        .build();
65    }
    }

package org.phillippko.ignite.internal.cli.optimise.command;

import static org.apache.ignite.internal.cli.commands.
    Options.Constants.NODE_NAME_OPTION;
5    import static org.apache.ignite.internal.cli.commands.
    Options.Constants.NODE_NAME_OPTION_DESC;
import static org.apache.ignite.internal.cli.commands.
    Options.Constants.NODE_NAME_OPTION_SHORT;

import jakarta.inject.Inject;
import java.util.concurrent.Callable;
10    import org.apache.ignite.internal.cli.commands.BaseCommand;
import org.apache.ignite.internal.cli.commands.cluster.
    ClusterUrlMixin;
import org.apache.ignite.internal.cli.commands.questions.
    ConnectToClusterQuestion;
import org.apache.ignite.internal.cli.core.call.
    CallExecutionPipeline;
import org.apache.ignite.internal.cli.core.exception.handler
    .ClusterNotInitializedExceptionHandler;
15    import org.apache.ignite.internal.cli.core.flow.builder.
    Flows;
import org.phillippko.ignite.internal.cli.optimise.call.
    RunBenchmarkCall;
import org.phillippko.ignite.internal.cli.optimise.call.
    RunBenchmarkCallInput;
import picocli.CommandLine.Command;
import picocli.CommandLine.Mixin;
20    import picocli.CommandLine.Option;
import picocli.CommandLine.Parameters;

@Command(name = "runBenchmark", description = "Runs
    benchmark")
public class RunBenchmarkReplCommand extends BaseCommand
    implements Runnable {

```

```

25  /** Node URL option. */
    @Mixin
    private ClusterUrlMixin clusterUrlMixin;

    @Inject
30  private RunBenchmarkCall call;

    @Parameters(
        arity = "0..1",
        index = "0",
35  description = "Path to the file containing SQL queries to
        run during benchmark."
    )
    private String benchmarkFilePath;

    @Mixin
40  private NodeNameMixin nodeName;

    @Option(names = "--iterations", description = "Number of
        iterations to run the benchmark.", defaultValue = "1")
    private int iterations;

45  @Option(names = "--values", description = "Number of
        values to run the benchmark.")
    private Long values;

    @Option(names = "--profile", description = "Storage
        profile to use for the benchmark.", defaultValue = "
        default")
50  private String profile;

    @Inject
    private ConnectToClusterQuestion question;

    @Override
55  public void run() {
        runFlow(question.askQuestionIfNotConnected(
            clusterUrlMixin.getClusterUrl())
            .map(this::buildCallInput)
            .then(Flows.fromCall(call))
            .exceptionHandler(ClusterNotInitializedExceptionHandler.
                createReplHandler("Cannot get results"))
60  .print()
        );
    }
}

```

```

        private RunBenchmarkCallInput buildCallInput(String url) {
65         return RunBenchmarkCallInput.builder()
            .setClusterUrl(url)
            .setBenchmarkFilePath(benchmarkFilePath)
            .setNodeName(nodeName.nodeName())
            .setIterations(iterations)
70         .setValues(values)
            .setProfile(profile)
            .build();
        }
    }

package org.phillippko.ignite.internal.cli.optimise.command;

import static org.apache.ignite.internal.cli.commands.
    Options.Constants.NODE_NAME_OPTION;
5 import static org.apache.ignite.internal.cli.commands.
    Options.Constants.NODE_NAME_OPTION_DESC;
import static org.apache.ignite.internal.cli.commands.
    Options.Constants.NODE_NAME_OPTION_SHORT;
import static org.apache.ignite.internal.cli.commands.
    Options.Constants.WRITE_INTENSIVE_OPTION;

import jakarta.inject.Inject;
10 import java.util.concurrent.Callable;
import org.phillippko.ignite.internal.cli.optimise.call.
    RunOptimiseCall;
import org.phillippko.ignite.internal.cli.optimise.call.
    RunOptimiseCallInput;
import org.apache.ignite.internal.cli.commands.BaseCommand;
import org.apache.ignite.internal.cli.commands.cluster.
    ClusterUrlMixin;
15 import org.apache.ignite.internal.cli.core.call.
    CallExecutionPipeline;
import picocli.CommandLine.Command;
import picocli.CommandLine.Mixin;
import picocli.CommandLine.Option;

20 @Command(name = "runOptimise", description = "Optimise
    cluster configuration.")
public class RunOptimiseCommand extends BaseCommand
    implements Callable<Integer> {
    /** Node URL option. */
    @Mixin

```

```

25     private ClusterUrlMixin clusterUrlMixin;

    @Inject
    private RunOptimiseCall call;

    @Mixin
30     private OptimiseMixin optimiseMixin;

    @Mixin
    private NodeNameMixin nodeName;

35     @Override
    public Integer call() {
        return runPipeline(CallExecutionPipeline.builder(call)
            .inputProvider(this::buildCallInput)
            .output(spec.commandLine().getOut())
40        );
    }

    private RunOptimiseCallInput buildCallInput() {
        return RunOptimiseCallInput.builder()
45        .setClusterUrl(clusterUrlMixin.getClusterUrl())
        .setWriteIntensive(optimiseMixin.writeIntensive())
        .setTunerTimeout(optimiseMixin.tunerTimeout())
        .setNodeName(nodeName.nodeName())
        .build();
50    }
}

package org.phillippko.ignite.internal.cli.optimise.command;

import static org.apache.ignite.internal.cli.commands.
    Options.Constants.NODE_NAME_OPTION;
5 import static org.apache.ignite.internal.cli.commands.
    Options.Constants.NODE_NAME_OPTION_DESC;
import static org.apache.ignite.internal.cli.commands.
    Options.Constants.NODE_NAME_OPTION_SHORT;
import static org.apache.ignite.internal.cli.commands.
    Options.Constants.WRITE_INTENSIVE_OPTION;

import jakarta.inject.Inject;
10 import java.util.concurrent.Callable;
import org.apache.ignite.internal.cli.commands.BaseCommand;
import org.apache.ignite.internal.cli.commands.cluster.
    ClusterUrlMixin;

```

```

import org.apache.ignite.internal.cli.commands.questions.
    ConnectToClusterQuestion;
import org.apache.ignite.internal.cli.core.call.
    CallExecutionPipeline;
15 import org.apache.ignite.internal.cli.core.exception.handler
    .ClusterNotInitializedExceptionHandler;
import org.apache.ignite.internal.cli.core.flow.builder.
    Flows;
import org.phillippko.ignite.internal.cli.optimise.call.
    RunOptimiseCall;
import org.phillippko.ignite.internal.cli.optimise.call.
    RunOptimiseCallInput;
import picocli.CommandLine.Command;
20 import picocli.CommandLine.Mixin;
import picocli.CommandLine.Option;

@Command(name = "runOptimise", description = "Optimise
    cluster configuration.")
public class RunOptimiseReplCommand extends BaseCommand
    implements Runnable {
25 /** Node URL option. */
    @Mixin
    private ClusterUrlMixin clusterUrlMixin;

    @Inject
30 private RunOptimiseCall call;

    @Mixin
    private NodeNameMixin nodeName;

35 @Mixin
    private OptimiseMixin optimiseMixin;

    @Inject
    private ConnectToClusterQuestion question;
40

    @Override
    public void run() {
        runFlow(question.askQuestionIfNotConnected(
            clusterUrlMixin.getClusterUrl())
            .map(this::buildCallInput)
45 .then(Flows.fromCall(call))
            .exceptionHandler(ClusterNotInitializedExceptionHandler.
                createReplHandler("Cannot get results"))
            .print())
    }

```

```

    );
}

50 private RunOptimiseCallInput buildCallInput(String url) {
    return RunOptimiseCallInput.builder()
        .setClusterUrl(url)
        .setWriteIntensive(optimiseMixin.writeIntensive())
55     .setTunerTimeout(optimiseMixin.tunerTimeout())
        .setNodeName(nodeName.nodeName())
        .build();
    }
}

apply from: "$rootDir/buildscripts/java-core.gradle"
apply from: "$rootDir/buildscripts/publishing.gradle"
apply from: "$rootDir/buildscripts/java-junit5.gradle"
5 apply from: "$rootDir/buildscripts/java-test-fixtures.gradle"
    "
    apply from: "$rootDir/buildscripts/java-integration-test.
        gradle"

description = 'ignite-optimiser'

10 dependencies {
    annotationProcessor project(':ignite-configuration-
        annotation-processor')
    annotationProcessor project(":ignite-network-annotation-
        processor")
    annotationProcessor libs.micronaut.inject.annotation.
        processor
15 annotationProcessor libs.auto.service

    implementation project(':ignite-core')
    implementation project(':ignite-configuration')
    implementation project(':ignite-configuration-api')
    implementation project(':ignite-configuration-presentation
        ')
20 implementation project(':ignite-failure-handler')
    implementation project(':ignite-network')
    implementation project(':ignite-raft-api')
    implementation project(':ignite-vault')
    implementation project(':ignite-network-api')
25 implementation project(':ignite-metastorage-api')
    implementation project(':ignite-schema')
    implementation project(':ignite-storage-api')

```

```

implementation project(':ignite-sql-engine')
implementation project(':ignite-page-memory')
30 implementation project(':ignite-storage-rocksdb')
implementation project(':ignite-metastorage')
implementation project(':ignite-rocksdb-common')
implementation project(':ignite-storage-api')
implementation project(':ignite-security')
35 implementation project(':ignite-configuration-root')
implementation project(':ignite-system-disaster-recovery-
    api')
implementation libs.jetbrains.annotations
implementation libs.fastutil.core
implementation libs.auto.service.annotations
40

testImplementation project(':ignite-core')
testImplementation project(':ignite-configuration')
testImplementation project(':ignite-network')
testImplementation project(':ignite-system-disaster-
    recovery')
45 testImplementation testFixtures(project(':ignite-core'))
testImplementation testFixtures(project(':ignite-
    configuration'))
testImplementation testFixtures(project(':ignite-network')
    )
testImplementation testFixtures(project(':ignite-vault'))

50 testImplementation libs.hamcrest.core
testImplementation libs.mockito.junit

testFixturesImplementation project(':ignite-core')
testFixturesImplementation project(':ignite-failure-
    handler')
55 testFixturesImplementation project(':ignite-raft')
testFixturesImplementation project(':ignite-raft-api')
testFixturesImplementation project(':ignite-storage-api')
testFixturesImplementation project(':ignite-vault')
testFixturesImplementation project(':ignite-security')
60 testFixturesImplementation project(':ignite-system-
    disaster-recovery-api')
testFixturesImplementation project(':ignite-system-
    disaster-recovery')
testFixturesImplementation testFixtures(project(':ignite-
    core'))
testFixturesImplementation testFixtures(project(':ignite-
    configuration'))

```

```

testFixturesImplementation testFixtures(project(':ignite-
network'))
65 testFixturesImplementation testFixtures(project(':ignite-
metrics'))
testFixturesImplementation testFixtures(project(':ignite-
raft'))
testFixturesImplementation testFixtures(project(':ignite-
failure-handler'))
testFixturesImplementation libs.jetbrains.annotations
testFixturesImplementation libs.hamcrest.core
70
integrationTestAnnotationProcessor libs.micronaut.inject.
annotation.processor
integrationTestImplementation project(':ignite-rest')
integrationTestImplementation project(':ignite-raft')
integrationTestImplementation project(':ignite-raft-api')
75 integrationTestImplementation project(':ignite-catalog')
integrationTestImplementation project(':ignite-runner')
integrationTestImplementation project(':ignite-system-
disaster-recovery')
integrationTestImplementation testFixtures(project)
integrationTestImplementation testFixtures(project(':
ignite-core'))
80 integrationTestImplementation testFixtures(project(':
ignite-configuration'))
integrationTestImplementation testFixtures(project(':
ignite-network'))
integrationTestImplementation testFixtures(project(':
ignite-runner'))
integrationTestImplementation testFixtures(project(':
ignite-metrics:'))
integrationTestImplementation testFixtures(project(':
ignite-raft'))
85 integrationTestImplementation libs.awaitility
integrationTestImplementation libs.jetbrains.annotations
}

package org.phillippko.ignite.internal.optimiser;

import org.jetbrains.annotations.Nullable;
5
public interface BenchmarkRunner {
    String runBenchmark(@Nullable String benchmarkFilePath,
        int iterations, @Nullable Long values, String profile);
}

```

```

package org.phillippko.ignite.internal.optimiser;

import static java.lang.System.currentTimeMillis;

5
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.List;
import java.util.UUID;
10
import org.apache.ignite.internal.logger.IgniteLogger;
import org.apache.ignite.internal.logger.Loggers;
import org.apache.ignite.sql.IgniteSql;
import org.apache.ignite.sql.ResultSet;
import org.apache.ignite.sql.SqlException;
15
import org.apache.ignite.sql.SqlRow;
import org.jetbrains.annotations.Nullable;

public class BenchmarkRunnerImpl implements BenchmarkRunner
{
    private static final IgniteLogger LOG = Loggers.forClass(
        BenchmarkRunnerImpl.class);
20
    private final IgniteSql sql;

    public BenchmarkRunnerImpl(IgniteSql sql) {
        this.sql = sql;
    }

25

    @Override
    public String runBenchmark(
        @Nullable String benchmarkFilePath,
30
        int iterations,
        @Nullable Long values,
        String profile
    ) throws SqlException {
        StringBuilder resultBuilder = new StringBuilder();

35
        try {
            if (benchmarkFilePath != null) {
                runBenchmark(benchmarkFilePath, iterations, profile,
                    resultBuilder);
            } else {
40
                runBenchmark(iterations, values, profile,
                    resultBuilder);
            }
        } catch (Exception e) {

```

```

        LOG.error("Error running a benchmark", e);
45     return "Error running a benchmark: " + e.getMessage();
    }

    LOG.info(resultBuilder.toString());

50     return resultBuilder.toString();
}

private void runBenchmark(
    String benchmarkFilePath,
55     int iterations,
    String profile,
    StringBuilder resultBuilder
) throws Exception {
    LOG.info("Benchmarking " + benchmarkFilePath + "...");
60

    Path path = Path.of(benchmarkFilePath);

    List<String> statements = Files.readAllLines(path);

65     for (int i = 1; i <= iterations; i++) {
        long atStart = currentTimeMillis();

        for (String statement : statements) {
70             executeSql(statement);
        }

        resultBuilder.append("Iteration ").append(i).append("
            finished in ").append(currentTimeMillis() - atStart
            ).append("MS\n");
    }
}

75
private void runBenchmark(int iterations, long values,
    String profile, StringBuilder resultBuilder) throws
    SQLException {
    LOG.info("Benchmarking profile " + profile + " inserting
        " + values + " values...");

    executeSql(String.format("CREATE ZONE IF NOT EXISTS
        ZONE_%s WITH STORAGE_PROFILES='%s'", profile, profile
        ));
80

```

```

        executeSql("DROP TABLE IF EXISTS test_table");

        executeSql("CREATE TABLE test_table (id VARCHAR PRIMARY
            KEY) ZONE ZONE_" + profile);

85     executeSql("DELETE FROM test_table");

        String insert = "INSERT INTO test_table (id) VALUES (?)
            ";

        for (int i = 1; i <= iterations; i++) {
90             long atStart = currentTimeMillis();

            for (int value = 0; value < values; value++) {
                executeSql(insert, UUID.randomUUID().toString());
            }

95             resultBuilder.append("Iteration ").append(i).append("
                finished in ").append(currentTimeMillis() - atStart
                    ).append("MS\n");
        }
    }

100     private void executeSql(String query, Object... params) {
        try (ResultSet<SqlRow> ignored = sql.execute(null, query
            , params)) {
        }
    }
}

package org.phillippko.ignite.internal.optimiser;

import java.util.UUID;
5   import org.apache.ignite.internal.network.NetworkMessage;
import org.apache.ignite.internal.network.annotations.
    Transferable;

    @Transferable(OptimiserMessageGroup.OPTIMISE_MESSAGE_TYPE)
    interface OptimiseMessage extends NetworkMessage {
10         boolean writeIntensive();

        int tunerTimeout();

        UUID id();
15     }

```

```

@@ -0,0 +1,5 @@
package org.phillippko.ignite.internal.optimiser;

5 public interface OptimiseRunner {
    String getIssues(boolean writeIntensive, int tunerTimeout)
        ;
}

package org.phillippko.ignite.internal.optimiser;

import java.io.IOException;
5 import java.lang.management.ManagementFactory;
import java.lang.management.MemoryUsage;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.ArrayList;
10 import java.util.List;
import java.util.concurrent.TimeUnit;
import java.util.stream.Collectors;
import org.apache.ignite.configuration.NamedListView;
import org.apache.ignite.internal.configuration.
    ConfigurationRegistry;
15 import org.apache.ignite.internal.logger.IgniteLogger;
import org.apache.ignite.internal.logger.Loggers;
import org.apache.ignite.internal.pagememory.configuration.
    schema.PersistentPageMemoryProfileView;
import org.apache.ignite.internal.pagememory.configuration.
    schema.VolatilePageMemoryProfileView;
import org.apache.ignite.internal.schema.configuration.
    GcConfiguration;
20 import org.apache.ignite.internal.schema.configuration.
    GcExtensionConfiguration;
import org.apache.ignite.internal.schema.configuration.
    LowWatermarkConfiguration;
import org.apache.ignite.internal.storage.configurations.
    StorageExtensionConfiguration;
import org.apache.ignite.internal.storage.configurations.
    StorageProfileView;
import org.apache.ignite.internal.storage.rocksdb.
    configuration.schema.RocksDbProfileView;
25

public class OptimiseRunnerImpl implements OptimiseRunner {
    private static final IgniteLogger LOG = Loggers.forClass(
        OptimiseRunnerImpl.class);

```

```

private static final String OPENTUNER_COMMAND = "C:/
    IdeaProjects/apachetuner -timeToRun=%d";
30

private static final Path OPENTUNER_RESULT_PATH = Path.of
    ("C:/tmp/opentuner");

private final ConfigurationRegistry
    clusterConfigurationRegistry;
private final ConfigurationRegistry
    nodeConfigurationRegistry;
35

public OptimiseRunnerImpl(ConfigurationRegistry
    clusterConfigurationRegistry, ConfigurationRegistry
    nodeConfigurationRegistry) {
    this.clusterConfigurationRegistry =
        clusterConfigurationRegistry;
    this.nodeConfigurationRegistry =
        nodeConfigurationRegistry;
40
}

@Override
public String getIssues(boolean writeIntensive, int
    tunerTimeout) {
    LOG.info("Running optimisation");

45
    List<String> issues = new ArrayList<>();
    MemoryUsage heapMemoryUsage = ManagementFactory.
        getMemoryMXBean().getHeapMemoryUsage();
    double freeMemory = Runtime.getRuntime().totalMemory() -
        Runtime.getRuntime().freeMemory();
    boolean closeToMaxMemory = freeMemory / Runtime.
        getRuntime().maxMemory() > 0.9;

50
    NamedListView<StorageProfileView> profiles =
        nodeConfigurationRegistry.getConfiguration(
            StorageExtensionConfiguration.KEY).storage()
            .profiles().value();

    if (closeToMaxMemory) {
        issues.add("More than 90% of maximum memory is used,
            increase -Xmx JVM option");
55
    } else {
        issues.add("Maximum memory OK");
    }
}

```

```

GcConfiguration gcConfig = clusterConfigurationRegistry.
    getConfiguration(GcExtensionConfiguration.KEY).gc();

60
if (closeToMaxMemory && gcConfig.threads().value() <
    Runtime.getRuntime().availableProcessors() / 2) {
    issues.add(
        "Less than half of available threads designated for GC
        and memory usage is high, increase gcConfiguration
        .threads, current: "
        + gcConfig.threads());
65
    } else {
        issues.add("GC threads OK");
    }

LowWatermarkConfiguration lowWatermarkConfiguration =
    gcConfig.lowWatermark();
70
if (lowWatermarkConfiguration.dataAvailabilityTime().
    value() > 1_000_000) {
    issues.add("Low watermark data availability is too
        high, causing extra disk usage: "
        + lowWatermarkConfiguration.dataAvailabilityTime());
    } else {
        issues.add("Low watermark data availability OK");
75
    }

if (lowWatermarkConfiguration.updateInterval().value() >
    lowWatermarkConfiguration.dataAvailabilityTime().
    value() / 2) {
    issues.add(
        "Low watermark update interval is too high [
        updateInterval=" +
80
        lowWatermarkConfiguration.updateInterval() +
        ", dataAvailabilityTime=" +
        lowWatermarkConfiguration.dataAvailabilityTime() + "]"
    );
    } else {
85
        issues.add("Low watermark update interval OK");
    }

long volatileDataRegionsSize = profiles.stream()
    .filter(VolatilePageMemoryProfileView.class::isInstance)
90
    .map(VolatilePageMemoryProfileView.class::cast)
    .mapToLong(VolatilePageMemoryProfileView::maxSize)
    .sum();

```

```

    if (volatileDataRegionsSize > heapMemoryUsage.getMax())
    {
95      issues.add("Sum of volatile data regions exceeds max
        heap size in bytes ("
        + heapMemoryUsage.getMax() + "): " +
        volatileDataRegionsSize);
    } else {
        issues.add("Volatile data regions size OK");
    }
100
    if (writeIntensive) {
        String persistentProfiles = profiles.stream()
            .filter(PersistentPageMemoryProfileView.class::
                isInstance)
            .map(StorageProfileView::name).collect(Collectors.
                joining());
105

        if (!persistentProfiles.isEmpty()) {
            issues.add("For write-intensive workloads RocksDB
                should be used instead page memory in these
                profiles: "
                + persistentProfiles);
        }
110    } else {
        String rocksDbProfiles = profiles.stream()
            .filter(RocksDbProfileView.class::isInstance)
            .map(StorageProfileView::name).collect(Collectors.
                joining());
115

        if (!rocksDbProfiles.isEmpty()) {
            issues.add(
                "For read-intensive workloads page memory should be
                used instead of RocksDB in these profiles: " +
                rocksDbProfiles
            );
        }
120    }

    String opentunerIssues = getIssuesWithOpentuner(
        tunerTimeout);

    if (!opentunerIssues.isEmpty()) {
125        issues.add(opentunerIssues);
    }

```

```

        String result = String.join("; ", issues);

130     LOG.info(result);

        return result;
    }

135     private static String getIssuesWithOpentuner(int
        tunerTimeout) {
        try {
            ProcessBuilder builder = new ProcessBuilder(String.
                format(OPENTUNER_COMMAND, tunerTimeout));

            if (builder.start().waitFor(tunerTimeout + 20,
                TimeUnit.SECONDS)) {
140                return Files.readString(OPENTUNER_RESULT_PATH);
            } else {
                return "Opentuner didn't finish in time";
            }

145        } catch (IOException | InterruptedException e) {
            LOG.error("Couldn't run opentuner: " +
                OPENTUNER_COMMAND, e);

            return "Couldn't run opentuner: " + e.getMessage();
        }
150    }
}

package org.phillippko.ignite.internal.optimiser;

import static org.apache.ignite.internal.util.ByteUtils.
    stringFromBytes;
5  import static org.apache.ignite.internal.util.ByteUtils.
    stringToBytes;
import static org.apache.ignite.internal.util.
    CompletableFutures.nullCompletedFuture;

import java.util.UUID;
import java.util.concurrent.CompletableFuture;
10 import java.util.concurrent.CompletionException;
import java.util.concurrent.ExecutorService;
import org.apache.ignite.internal.lang.ByteArray;
import org.apache.ignite.internal.manager.ComponentContext;
import org.apache.ignite.internal.manager.IgniteComponent;

```

```

15 import org.apache.ignite.internal.metastorage.
    MetaStorageManager;
import org.apache.ignite.internal.network.MessagingService;
import org.apache.ignite.internal.network.NetworkMessage;
import org.apache.ignite.internal.network.TopologyService;
import org.apache.ignite.network.ClusterNode;
20 import org.jetbrains.annotations.Nullable;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class OptimiserManager implements IgniteComponent {
25     public static final String RESULT_PREFIX = "optimiser.
        result.";
    private static final Logger LOG = LoggerFactory.getLogger(
        OptimiserManager.class);
    private final ExecutorService threadPool;
    private final MessagingService messagingService;
    private final MetaStorageManager metastorageManager;
30     private final OptimiserMessageFactory messageFactory;
    private final TopologyService topologyService;
    private final OptimiseRunner optimiseRunner;
    private final BenchmarkRunner benchmarkRunner;

35     public OptimiserManager(
        MetaStorageManager metastorageManager,
        ExecutorService threadPool,
        MessagingService messagingService,
        OptimiserMessageFactory messageFactory,
40     TopologyService topologyService,
        OptimiseRunner optimiseRunner,
        BenchmarkRunner benchmarkRunner
    ) {
        this.messageFactory = messageFactory;
45     this.optimiseRunner = optimiseRunner;
        this.benchmarkRunner = benchmarkRunner;
        this.topologyService = topologyService;
        this.threadPool = threadPool;
        this.messagingService = messagingService;
50     this.metastorageManager = metastorageManager;
    }

    public CompletableFuture<UUID> optimise(@Nullable String
        nodeName, boolean writeIntensive, int tunerTimeout) {
        LOG.info("Running optimisation");
55

```

```

        UUID id = UUID.randomUUID();

        ClusterNode targetNode = getNode(nodeName);

60    OptimiseMessage message = messageFactory.optimiseMessage
        ()
        .writeIntensive(writeIntensive)
        .id(id)
        .tunerTimeout(tunerTimeout)
        .build();

65    return messagingService.send(targetNode, message)
        .thenApply(ignored -> id);
    }

70    public CompletableFuture<UUID> runBenchmark(
        @Nullable String nodeName,
        @Nullable String benchmarkFilePath,
        int iterations,
        @Nullable Long values,
75    String profile
    ) {
        LOG.info("Running a benchmark");

        UUID id = UUID.randomUUID();

80    ClusterNode targetNode = getNode(nodeName);

        NetworkMessage message = messageFactory.
            runBenchmarkMessage()
            .benchmarkFileName(benchmarkFilePath)
85    .id(id)
            .iterations(iterations)
            .values(values)
            .profile(profile)
            .build();

90    return messagingService.send(targetNode, message)
        .thenApply(ignored -> id);
    }

95    public CompletableFuture<String> optimisationResult(UUID
        id) {
        return metastorageManager.get(ByteArray.fromString(
            RESULT_PREFIX + id))
    }

```

```

        .handle((res, e) -> {
            LOG.info("Getting info about " + id);

            if (e != null) {
                LOG.error("Couldn't get result: ", e);

                throw new CompletionException(e);
            }

            if (res.empty() || res.tombstone()) {
                throw new CompletionException(new
                    IllegalArgumentException("Incorrect ID"));
            }

            return "Optimisation result: " + stringFromBytes(res.
                value());
        });
    }

    @Override
    public CompletableFuture<Void> startAsync(ComponentContext
        componentContext) {
        messagingService.addMessageHandler(OptimiserMessageGroup
            .class, this::handleMessage);

        return nullCompletedFuture();
    }

    @Override
    public CompletableFuture<Void> stopAsync(ComponentContext
        componentContext) {
        return nullCompletedFuture();
    }

    private void runOptimiseInternal(UUID id, boolean
        writeIntensive, int tunerTimeout) {
        ByteArray key = ByteArray.fromString(RESULT_PREFIX + id)
            ;

        metastorageManager.put(key, "STARTED".getBytes())
            .thenApplyAsync((v) -> optimiseRunner.getIssues(
                writeIntensive, tunerTimeout), threadPool)
            .thenAccept((issues) -> metastorageManager.put(key,
                stringToBytes(issues)))
            .exceptionally(e -> {

```

```

        LOG.error("Error while optimising: ", e);

135     metastorageManager.put(key, stringToBytes("
        Optimisation FAILED: " + e.getMessage()));

        return null;
    });
}

140 private void runBenchmarkInternal(UUID id, @Nullable
    String benchmarkFilePath, int iterations, @Nullable
    Long values, String profile) {
    ByteArray key = ByteArray.fromString(RESULT_PREFIX + id)
        ;

    metastorageManager.put(key, "Benchmark STARTED".getBytes
        ())
145     .thenApplyAsync((v) -> benchmarkRunner.runBenchmark(
        benchmarkFilePath, iterations, values, profile),
        threadPool)
        .thenAccept((result) -> metastorageManager.put(key,
            stringToBytes(result)));
}

private ClusterNode getNode(@Nullable String nodeName) {
150     if (nodeName == null) {
        return topologyService.localMember();
    }

    ClusterNode node = topologyService.getByConsistentId(
        nodeName);
155     if (node == null) {
        throw new IllegalArgumentException("Node not found by
            name: " + nodeName);
    }

    return node;
160 }

private void handleMessage(NetworkMessage message,
    ClusterNode sender, @Nullable Long correlationId) {
    if (message instanceof RunBenchmarkMessage) {
165         RunBenchmarkMessage runBenchmarkMessage = (
            RunBenchmarkMessage) message;

```

```

        runBenchmarkInternal(
            runBenchmarkMessage.id(),
            runBenchmarkMessage.benchmarkFileName(),
170         runBenchmarkMessage.iterations(),
            runBenchmarkMessage.values(),
            runBenchmarkMessage.profile()
        );
    } else if (message instanceof OptimiseMessage) {
175         OptimiseMessage optimiseMessage = (OptimiseMessage)
            message;

        runOptimiseInternal(optimiseMessage.id(),
            optimiseMessage.writeIntensive(), optimiseMessage.
            tunerTimeout());
    }
}
180 }

package org.phillippko.ignite.internal.optimiser;

import static org.phillippko.ignite.internal.optimiser.
    OptimiserMessageGroup.GROUP_TYPE;
5
import org.apache.ignite.internal.network.annotations.
    MessageGroup;

@MessageGroup(groupType = GROUP_TYPE, groupName = "
    OptimiserMessage")
public interface OptimiserMessageGroup {
10     short GROUP_TYPE = 17;
    short RUN_BENCHMARK_TYPE = 0;
    short OPTIMISE_MESSAGE_TYPE = 1;
}

package org.phillippko.ignite.internal.optimiser;

import java.util.UUID;
5
import org.apache.ignite.internal.network.NetworkMessage;
import org.apache.ignite.internal.network.annotations.
    Transferable;
import org.jetbrains.annotations.Nullable;

@Transferable(OptimiserMessageGroup.RUN_BENCHMARK_TYPE)
10 interface RunBenchmarkMessage extends NetworkMessage {
    @Nullable String benchmarkFileName();

```

```

        UUID id();

15    int iterations();

        String profile();

        @Nullable Long values();
20    }

package org.apache.ignite.internal.rest.api.optimise;

import io.micronaut.http.annotation.Body;
5    import io.micronaut.http.annotation.Controller;
import io.micronaut.http.annotation.Get;
import io.micronaut.http.annotation.Post;
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.media.Content;
10    import io.swagger.v3.oas.annotations.media.Schema;
import io.swagger.v3.oas.annotations.responses.ApiResponse;
import io.swagger.v3.oas.annotations.tags.Tag;
import java.util.UUID;
import java.util.concurrent.CompletableFuture;
15    import org.apache.ignite.internal.rest.api.Problem;
import org.apache.ignite.internal.rest.constants.MediaType;

@Controller("/management/v1/optimise")
@Tag(name = "optimise")
20    public interface OptimiseApi {
        @Post("runBenchmark")
        @Operation(operationId = "runBenchmark", summary = "Run
            benchmark", description = "Runs benchmark.")
        @ApiResponse(responseCode = "200", description = "
            Benchmark started.",
            content = @Content(mediaType = MediaType.APPLICATION_JSON,
                schema = @Schema(implementation = UUID.class)))
25    @ApiResponse(responseCode = "500", description = "Internal
            error.",
            content = @Content(mediaType = MediaType.PROBLEM_JSON,
                schema = @Schema(implementation = Problem.class)))
        CompletableFuture<UUID> runBenchmark(
            @Body RunBenchmarkRequest runBenchmarkRequest
        );
30

        @Post("optimise")

```

```

35  @Operation(operationId = "optimise", summary = "Run
    optimisation", description = "Runs optimisation.")
    @ApiResponse(responseCode = "200", description = "
        Optimisation started.",
        content = @Content(mediaType = MediaType.APPLICATION_JSON,
            schema = @Schema(implementation = UUID.class)))
40  @ApiResponse(responseCode = "500", description = "Internal
        error.",
        content = @Content(mediaType = MediaType.PROBLEM_JSON,
            schema = @Schema(implementation = Problem.class)))
    CompletableFuture<UUID> optimise(@Body
        RunOptimisationRequest runOptimisationRequest);

    @Get("optimisationResult")
40  @Operation(operationId = "optimisationResult", summary = "
        Get result", description = "Returns optimisation or
        benchmark result.")
    @ApiResponse(responseCode = "200", description = "Got
        result.",
        content = @Content(mediaType = MediaType.APPLICATION_JSON,
            schema = @Schema(implementation = String.class)))
    @ApiResponse(responseCode = "500", description = "Internal
        error.",
        content = @Content(mediaType = MediaType.PROBLEM_JSON,
            schema = @Schema(implementation = Problem.class)))
45  CompletableFuture<String> optimisationResult(UUID id);
}

package org.apache.ignite.internal.rest.api.optimise;

import com.fasterxml.jackson.annotation.JsonCreator;
5  import com.fasterxml.jackson.annotation.JsonGetter;
import com.fasterxml.jackson.annotation.JsonProperty;
import io.swagger.v3.oas.annotations.media.Schema;
import org.apache.ignite.internal.tostring.S;
import org.jetbrains.annotations.Nullable;

10  @Schema(description = "Run benchmark.")
    public class RunBenchmarkRequest {
        private final @Nullable String benchmarkFilePath;
        private final @Nullable String nodeName;
15  private final int iterations;
        private final @Nullable Long values;
        private final String profile;

```

```

20  /** Constructor. */
    @JsonCreator
    public RunBenchmarkRequest(
        @JsonProperty("nodeName") @Nullable String nodeName,
        @JsonProperty("benchmarkFilePath") @Nullable String
            benchmarkFilePath,
        @JsonProperty("iterations") int iterations,
25  @JsonProperty("values") @Nullable Long values,
        @JsonProperty("profile") String profile
    ) {
        this.benchmarkFilePath = benchmarkFilePath;
        this.nodeName = nodeName;
30  this.iterations = iterations;
        this.values = values;
        this.profile = profile;
    }

35  @JsonGetter
    public @Nullable String benchmarkFilePath() {
        return benchmarkFilePath;
    }

40  @JsonGetter
    @Nullable
    public String nodeName() {
        return nodeName;
    }

45  @JsonGetter
    public int iterations() {
        return iterations;
    }

50  @JsonGetter
    public @Nullable Long values() {
        return values;
    }

55  @JsonGetter
    public String profile() {
        return profile;
    }

60  @Override
    public String toString() {

```

```

        return S.toString(this);
    }
65 }

package org.apache.ignite.internal.rest.api.optimise;

import com.fasterxml.jackson.annotation.JsonCreator;
5 import com.fasterxml.jackson.annotation.JsonGetter;
import com.fasterxml.jackson.annotation.JsonProperty;
import io.swagger.v3.oas.annotations.media.Schema;
import org.apache.ignite.internal.tostring.S;
import org.jetbrains.annotations.Nullable;
10

@Schema(description = "Run optimisation.")
public class RunOptimisationRequest {
    @Schema(description = "If target configuration should be
        optimised for write-intensive use-cases.")
    private final boolean writeIntensive;
15 @Schema(description = "Name of the node to run
        optimisation on.")
    private final @Nullable String nodeName;

    @Schema(description = "How long should OpenTuner run in
        milliseconds.")
    private final int tunerTimeout;
20

    /** Constructor. */
    @JsonCreator
    public RunOptimisationRequest(
        @JsonProperty("nodeName") @Nullable String nodeName,
25 @JsonProperty("writeIntensive") boolean writeIntensive,
        @JsonProperty("tunerTimeout") int tunerTimeout
    ) {
        this.writeIntensive = writeIntensive;
        this.nodeName = nodeName;
30 this.tunerTimeout = tunerTimeout;
    }

    @JsonGetter
    public boolean writeIntensive() {
35 return writeIntensive;
    }

    @JsonGetter
    public int tunerTimeout() {

```

```

40         return tunerTimeout;
    }

    @JsonGetter
    @Nullable
45     public String nodeName() {
        return nodeName;
    }

    @Override
50     public String toString() {
        return S.toString(this);
    }
}

package org.apache.ignite.internal.rest.optimiser;

import io.micronaut.http.annotation.Controller;
5     import java.util.UUID;
    import java.util.concurrent.CompletableFuture;
    import org.apache.ignite.internal.rest.ResourceHolder;
    import org.apache.ignite.internal.rest.api.optimise.
        OptimiseApi;
    import org.apache.ignite.internal.rest.api.optimise.
        RunOptimisationRequest;
10    import org.apache.ignite.internal.rest.api.optimise.
        RunBenchmarkRequest;
    import org.phillippko.ignite.internal.optimiser.
        OptimiserManager;

    @Controller("/management/v1/optimise")
    public class OptimiseController implements OptimiseApi,
        ResourceHolder {
15        private OptimiserManager optimiserManager;

        public OptimiseController(OptimiserManager
            optimiserManager) {
            this.optimiserManager = optimiserManager;
        }
20

        @Override
        public CompletableFuture<UUID> runBenchmark(
            RunBenchmarkRequest runBenchmarkRequest) {
            return optimiserManager.runBenchmark(
                runBenchmarkRequest.nodeName(),

```

```

25     runBenchmarkRequest.benchmarkFilePath(),
        runBenchmarkRequest.iterations(),
        runBenchmarkRequest.values(),
        runBenchmarkRequest.profile()
30     );
    }

    @Override
    public CompletableFuture<UUID> optimise(
        RunOptimisationRequest runOptimisationRequest) {
        return optimiserManager.optimise(
35         runOptimisationRequest.nodeName(),
            runOptimisationRequest.writeIntensive(),
            runOptimisationRequest.tunerTimeout()
        );
    }

40     @Override
    public CompletableFuture<String> optimisationResult(UUID
        id) {
        return optimiserManager.optimisationResult(id);
    }

45     @Override
    public void cleanResources() {
        optimiserManager = null;
    }

50 }

package org.apache.ignite.internal.rest.optimiser;

import io.micronaut.context.annotation.Bean;
5 import io.micronaut.context.annotation.Factory;
import jakarta.inject.Singleton;
import org.apache.ignite.internal.rest.RestFactory;
import org.phillippko.ignite.internal.optimiser.
    OptimiserManager;

10 /**
 * Factory that defines beans required for the rest module.
 */
@Factory
public class OptimiserFactory implements RestFactory {
15     private OptimiserManager optimiserManager;

```

```

    public OptimiserFactory(OptimiserManager optimiserManager)
    {
        this.optimiserManager = optimiserManager;
    }

    @Bean
    @Singleton
    public OptimiserManager optimiserManager() {
        return optimiserManager;
    }

    @Override
    public void cleanResources() {
        optimiserManager = null;
    }
}

package org.apache.ignite.internal.benchmark;

import java.io.FileWriter;
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.AtomicInteger;
import org.apache.ignite.internal.lang.
    IgniteSystemProperties;
import org.apache.ignite.table.KeyValueView;
import org.apache.ignite.table.Tuple;
import org.openjdk.jmh.annotations.Benchmark;
import org.openjdk.jmh.annotations.BenchmarkMode;
import org.openjdk.jmh.annotations.Fork;
import org.openjdk.jmh.annotations.Measurement;
import org.openjdk.jmh.annotations.Mode;
import org.openjdk.jmh.annotations.OutputTimeUnit;
import org.openjdk.jmh.annotations.Param;
import org.openjdk.jmh.annotations.Scope;
import org.openjdk.jmh.annotations.Setup;
import org.openjdk.jmh.annotations.State;
import org.openjdk.jmh.annotations.Threads;
import org.openjdk.jmh.annotations.Warmup;
import org.openjdk.jmh.runner.Runner;
import org.openjdk.jmh.runner.RunnerException;
import org.openjdk.jmh.runner.options.Options;

```

```

import org.openjdk.jmh.runner.options.OptionsBuilder;

30  /**
   * Benchmark for a single upsert operation via KV API with a
   * possibility to disable updates via RAFT and to storage.
   */
   @State(Scope.Benchmark)
35  @Fork(1)
   @Threads(1)
   @Warmup(iterations = 2, time = 2)
   @Measurement(iterations = 5, time = 2)
   @BenchmarkMode(Mode.AverageTime)
40  @OutputTimeUnit(TimeUnit.MICROSECONDS)
   public class UpsertKvBenchmark extends
       AbstractMultiNodeBenchmark {
       private final Tuple tuple = Tuple.create();

       private static KeyValueCollection<Tuple, Tuple> kvView;

45  private int batch = 10000;

       @Param({"false", "true"})
       private boolean fsync;

50  private int partitionCount = 8;

       private static final AtomicInteger counter = new
           AtomicInteger();

55  private static final ThreadLocal<Integer> gen =
       ThreadLocal.withInitial(() -> {
           int id = counter.getAndIncrement();
           return id * 20_000_000;
       });

60  @Override
       public void nodeSetUp() throws Exception {
           System.setProperty(IgniteSystemProperties.
               IGNITE_SKIP_REPLICATION_IN_BENCHMARK, "true");
           System.setProperty(IgniteSystemProperties.
               IGNITE_SKIP_STORAGE_UPDATE_IN_BENCHMARK, "true");
           super.nodeSetUp();
65  }

       /**

```

```

* Initializes the tuple.
*/
70 @Setup
public void setUp() {
    kvView = igniteImpl.tables().table(TABLE_NAME).
        keyValueView();
    for (int i = 1; i < 11; i++) {
75         tuple.set("field" + i, FIELD_VAL);
    }
}

/**
* Benchmark for KV upsert via embedded client.
80 */
@Benchmark
public void upsert() {
    List<CompletableFuture<Void>> futs = new ArrayList<>();

85     for (int i = 0; i < batch - 1; i++) {
        CompletableFuture<Void> fut = kvView.putAsync(null,
            Tuple.create().set("ycsb_key", nextId()), tuple);
        futs.add(fut);
    }

90     for (CompletableFuture<Void> fut : futs) {
        fut.join();
    }

    kvView.put(null, Tuple.create().set("ycsb_key", nextId())
95         ), tuple);
}

private int nextId() {
    int cur = gen.get() + 1;
    gen.set(cur);
100     return cur;
}

/**
* Benchmark's entry point.
105 */
public static void main(String[] args) throws
    RunnerException, IOException {
    Options opt = new OptionsBuilder()

```

```

        .include(".*" + UpsertKvBenchmark.class.getSimpleName()
            + ".*")
        .param("fsync", args[1])
110    .build();

    int averageTime = (int) new Runner(opt).run().stream().
        findFirst().get().getPrimaryResult().getScore();

    var fileWriter = new FileWriter(args[0]);
115

    fileWriter.write(averageTime + "");
    fileWriter.close();
}

120    @Override
    protected boolean fsync() {
        return fsync;
    }

125    @Override
    protected int nodes() {
        return 1;
    }

130    @Override
    protected int partitionCount() {
        return partitionCount;
    }

135    @Override
    protected int replicaCount() {
        return 1;
    }
}

```

Приложение 2

Исходный код тестов приложения

```

package org.apache.ignite.internal.cli.commands.optimise;

import static org.apache.ignite.internal.cli.commands.
    Options.Constants.CLUSTER_URL_OPTION;
5
import org.apache.ignite.internal.cli.CliIntegrationTest;
import org.junit.jupiter.api.Test;

public class ItRunBenchmarkCommandTest extends
    CliIntegrationTest {
10
    @Test
    public void testRunBenchmark() {
        execute("optimise", "runBenchmark", CLUSTER_URL_OPTION,
            NODE_URL, "benchmark.sql");

        assertErrOutputIsEmpty();
15
        assertOutputContains("Benchmark was started successfully
            with id ");
    }

    @Test
    public void testRunBenchmarkWithValues() {
20
        execute("optimise", "runBenchmark", CLUSTER_URL_OPTION,
            NODE_URL, "--values", "100");

        assertErrOutputIsEmpty();
        assertOutputContains("Benchmark was started successfully
            with id ");
    }
25
}

package org.apache.ignite.internal.cli.commands.optimise;

import static org.apache.ignite.internal.cli.commands.
    Options.Constants.CLUSTER_URL_OPTION;
5
import static org.apache.ignite.internal.cli.commands.
    Options.Constants.WRITE_INTENSIVE_OPTION;
import static org.assertj.core.api.Assertions.assertThat;

import org.apache.ignite.internal.Cluster;

```

```

import org.apache.ignite.internal.cli.CliIntegrationTest;
import org.junit.jupiter.api.Test;

public class ItRunOptimiseCommandTest extends
    CliIntegrationTest {
    @Test
    public void testRunOptimise() {
        execute("optimise", "runOptimise", CLUSTER_URL_OPTION, "
            http://localhost:" + Cluster.BASE_HTTP_PORT,
            WRITE_INTENSIVE_OPTION);

        assertErrOutputEmpty();

        assertOutput("Optimisation was started successfully with
            id ");
    }

    private void assertErrOutputEmpty() {
        assertThat(serr.toString())
            .as("Expected command error output to be empty")
            .isEmpty();
    }

    private void assertOutput(String expectedOutput) {
        assertThat(sout.toString())
            .as("Expected command output to contain: " +
                expectedOutput + " but was " + sout.toString())
            .contains(expectedOutput);
    }
}

package org.phillippko.ignite.internal.optimiser;

import static org.apache.ignite.internal.testframework.
    matchers.CompletableFutureMatcher.willBe;
import static org.apache.ignite.internal.testframework.
    matchers.CompletableFutureMatcher.
        willCompleteSuccessfully;
import static org.apache.ignite.internal.util.ByteUtils.
    stringToBytes;
import static org.hamcrest.MatcherAssert.assertThat;
import static org.hamcrest.Matchers.notNullValue;
import static org.mockito.Mockito.mock;
import static org.mockito.Mockito.times;
import static org.mockito.Mockito.verify;

```

```

import static org.mockito.Mockito.verifyNoMoreInteractions;
import static org.mockito.Mockito.when;
import static org.phillippko.ignite.internal.optimiser.
    OptimiserManager.RESULT_PREFIX;
15
import java.util.UUID;
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
20
import org.apache.ignite.internal.lang.ByteArray;
import org.apache.ignite.internal.manager.ComponentContext;
import org.apache.ignite.internal.metastorage.
    MetaStorageManager;
import org.apache.ignite.internal.network.MessagingService;
import org.apache.ignite.internal.network.TopologyService;
25
import org.apache.ignite.internal.testframework.
    BaseIgniteAbstractTest;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
30
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

@ExtendWith(MockitoExtension.class)
class OptimiserManagerTest extends BaseIgniteAbstractTest {
35
    private static final boolean WRITE_INTENSIVE = true;

    @Mock
    private TopologyService topologyService;

    @Mock
40
    private MessagingService messagingService;

    private final ComponentContext componentContext = new
        ComponentContext();

    private final OptimiserMessageFactory messageFactory = new
45
        OptimiserMessageFactory();

    private final OptimiseRunner optimiseRunner = mock(
        OptimiseRunner.class);

    private final BenchmarkRunner benchmarkRunner = mock(
        BenchmarkRunner.class);

```

```

50     private final ExecutorService threadPool = Executors.
        newSingleThreadExecutor();

    private final MetaStorageManager metastorageManager = mock
        (MetaStorageManager.class);

55     private static final String MOCK_OPTIMISER_RESULT = "mock
        optimiser result";

    private OptimiserManager manager;

    @BeforeEach
60     void init() {
        manager = new OptimiserManager(
            metastorageManager,
            threadPool,
            messagingService,
65            messageFactory,
            topologyService,
            optimiseRunner,
            benchmarkRunner
        );

70         when(optimiseRunner.getIssues(WRITE_INTENSIVE)).
            thenReturn(MOCK_OPTIMISER_RESULT);

        assertThat(manager.startAsync(componentContext),
            willCompleteSuccessfully());
    }

75     @AfterEach
    void cleanup() {
        threadPool.shutdown();

80         assertThat(manager.stopAsync(), willCompleteSuccessfully
            ());
    }

    @Test
    void runOptimise() {
85         CompletableFuture<UUID> optimise = manager.optimise(null
            , WRITE_INTENSIVE);

        // Вызов завершится успешно и вернет UUID

```

```

        assertThat(optimise, willBe(notNullValue()));

90    UUID id = optimise.join();

    // Имя узла не указано, должно быть отправлено на локаль
    // ный узел.
    verify(topologyService, times(1)).localMember();
    verifyNoMoreInteractions(topologyService);

95    verify(metastorageManager).put(new ByteArray(
        RESULT_PREFIX + id), stringToBytes("STARTED"));
    verify(metastorageManager).put(new ByteArray(
        RESULT_PREFIX + id), stringToBytes(
            MOCK_OPTIMISER_RESULT));
    }
}

package org.apache.ignite.internal.rest.cluster.optimiser;

import static java.util.concurrent.CompletableFuture.
    completedFuture;
5    import static org.apache.ignite.internal.rest.constants.
        HttpStatusCode.OK;
import static org.hamcrest.MatcherAssert.assertThat;
import static org.hamcrest.Matchers.is;
import static org.mockito.Mockito.mock;
import static org.mockito.Mockito.verify;
10    import static org.mockito.Mockito.when;

import io.micronaut.context.annotation.Bean;
import io.micronaut.context.annotation.Property;
import io.micronaut.context.annotation.Replaces;
15    import io.micronaut.http.HttpRequest;
import io.micronaut.http.HttpResponse;
import io.micronaut.http.client.HttpClient;
import io.micronaut.http.client.annotation.Client;
import io.micronaut.test.extensions.junit5.annotation.
    MicronautTest;
20    import jakarta.inject.Inject;
import java.util.UUID;
import org.apache.ignite.internal.rest.api.optimise.
    RunOptimisationRequest;
import org.apache.ignite.internal.rest.optimiser.
    OptimiserFactory;

```

```

import org.apache.ignite.internal.testframework.
    BaseIgniteAbstractTest;
25 import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.mockito.Mockito;
import org.phillippko.ignite.internal.optimiser.
    OptimiserManager;

30 @MicronautTest
@Property(name = "ignite.endpoints.filter-non-initialized",
    value = "false")
@Property(name = "micronaut.security.enabled", value = "
    false")
class OptimiserControllerTest extends BaseIgniteAbstractTest
{
    private static final String NODE_NAME = "node1";

35 @Inject
@Client("/management/v1/optimise/")
private HttpClient client;

    private static final OptimiserManager optimiserManager =
        mock(OptimiserManager.class);

    @BeforeEach
    void resetMocks() {
        Mockito.reset(optimiserManager);
45 }

    @Bean
    @Replaces(OptimiserFactory.class)
    OptimiserFactory optimiserFactory() {
50     return new OptimiserFactory(optimiserManager);
    }

    @Test
    void runsOptimise() {
55     UUID uuid = UUID.randomUUID();
        boolean writeIntensive = true;

        when(optimiserManager.optimise(NODE_NAME, writeIntensive
            )).thenReturn(completedFuture(uuid));

60     HttpRequest<RunOptimisationRequest> post = HttpRequest.
        POST("/optimise", RunOptimisationRequest.class)

```

```

        .body(new RunOptimisationRequest(NODE_NAME,
            writeIntensive));

        HttpResponse<Void> response = client.toBlocking().
            exchange(post);

65      assertThat(response.getStatus().getCode(), is(OK.code()))
            );
        verify(optimiserManager).optimise(NODE_NAME,
            writeIntensive);
    }

    @Test
70    void runsBenchmark() {
        UUID uuid = UUID.randomUUID();
        String benchmarkFilePath = "benchmarkPath";

        when(optimiserManager.runBenchmark(NODE_NAME,
            benchmarkFilePath)).thenReturn(completedFuture(uuid))
            ;

75      HttpRequest<String> post = HttpRequest.POST("/
            runBenchmark", String.class).body(benchmarkFilePath);

        HttpResponse<Void> response = client.toBlocking().
            exchange(post);

80      assertThat(response.getStatus().getCode(), is(OK.code()))
            );
        verify(optimiserManager).runBenchmark(NODE_NAME,
            benchmarkFilePath);
    }
}

```

Приложение 3

Конфигурации кластеров использованные при апробации

ПЗ.1. Исходная конфигурация

Файл cluster-config.conf, все опущенные настройки определяются значениями по умолчанию:

```

ignite {
  gc {
    lowWatermark {
5      dataAvailabilityTime=1000000
      updateInterval=700000
    }
    threads=4
  }
10 }

```

Файл ignite-config.conf, определяющий настройки узла. Опущенные настройки определяются значениями по умолчанию:

```

ignite {
  storage {
    profiles=[
5    {
      engine=aipersist
      name=default
    }
    {
10    engine=rocksdb
      name="rocksdb_profile"
    }
    {
      {
15      engine=aimem
      name="aimem_profile"
      maxSize=2073741824
    }
    ]
  }
20 }

```

Файл vars.env, определяющий настройки JVM:

```
#Run props
```

```

5  NODE_NAME=node1
   WORK_DIR=@IGNITE_WORK_DIR@
   CONFIG_FILE=@CONF_DIR@/@IGNITE_CONF_FILE@

   #JVM props
   JVM_MAX_MEM="1024M"
10  JVM_MIN_MEM="512M"
   JVM_GC="G1GC"
   JVM_G1HeapRegionSize="32M"
   JVM_GC_LOG_NAME="gc.log.$(date -u +%Y%m%d_%H%M%S)"
   JVM_GC_LOG_SIZE="100m"
15  JVM_GC_NUM_LOGS="10"
   ##For any additional users settings
   IGNITE3_EXTRA_JVM_ARGS=

```

П3.2. Итоговая конфигурация

Настройки кластера:

```

5  ignite {
   gc {
     lowWatermark {
       dataAvailabilityTime=600000
       updateInterval=300000
     }
     threads=10
   }
10 }

```

Настройки узла:

```

5  ignite {
   storage {
     profiles=[
       {
         engine=aipersist
         name=aipersist_profile
       }
       {
10      engine=rocksdb
         name=default
       }
     ]
   }
15 }

```

Файл `vars.env`, определяющий настройки JVM:

```
5  #Run props
   NODE_NAME=node1
   WORK_DIR=@IGNITE_WORK_DIR@
   CONFIG_FILE=@CONF_DIR@/@IGNITE_CONF_FILE@

   #JVM props
   JVM_MAX_MEM="16G"
10  JVM_MIN_MEM="8G"
   JVM_GC="G1GC"
   JVM_G1HeapRegionSize="32M"
   JVM_GC_LOG_NAME="gc.log.$(date -u +%Y%m%d_%H%M%S)"
   JVM_GC_LOG_SIZE="100m"
15  JVM_GC_NUM_LOGS="10"
   ##For any additional users settings
   IGNITE3_EXTRA_JVM_ARGS=
```